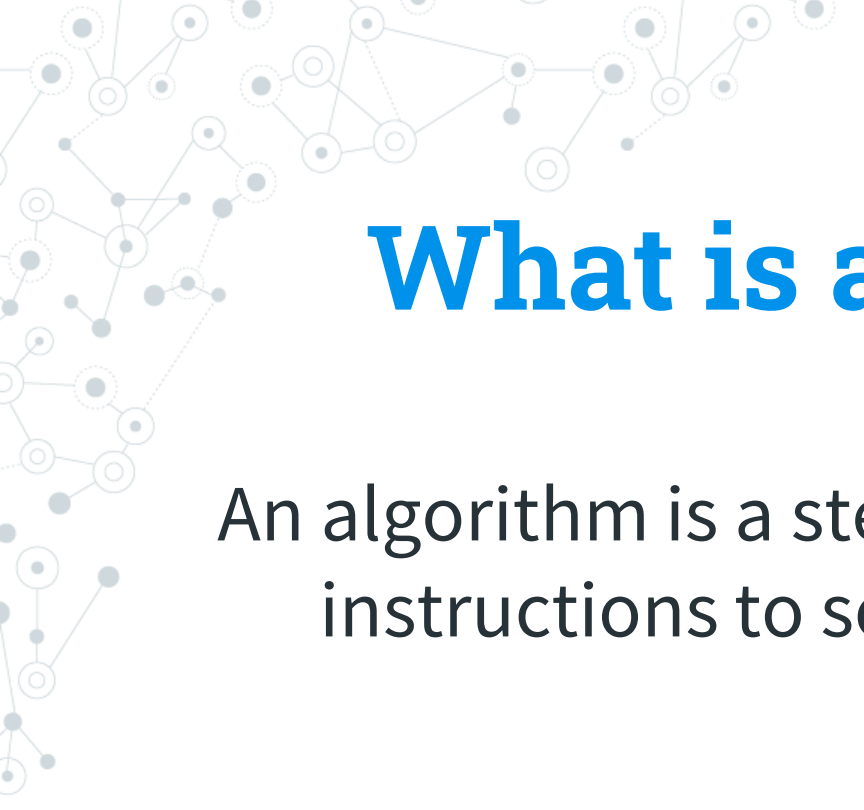




CSC 102– Discrete Structures I

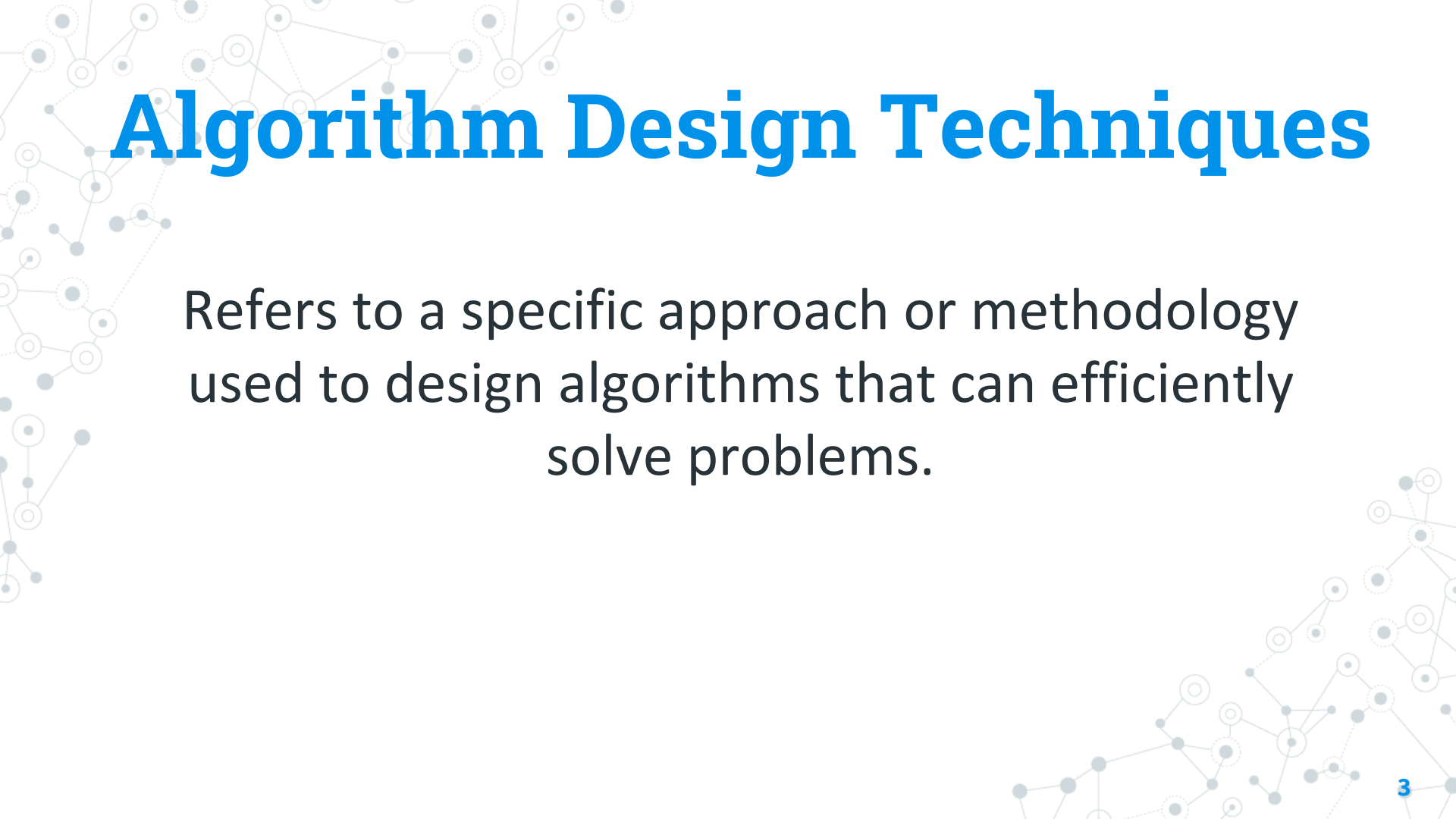
Algorithms





What is an algorithm ?

An algorithm is a step by step unambiguous instructions to solve a given problem.



Algorithm Design Techniques

Refers to a specific approach or methodology used to design algorithms that can efficiently solve problems.

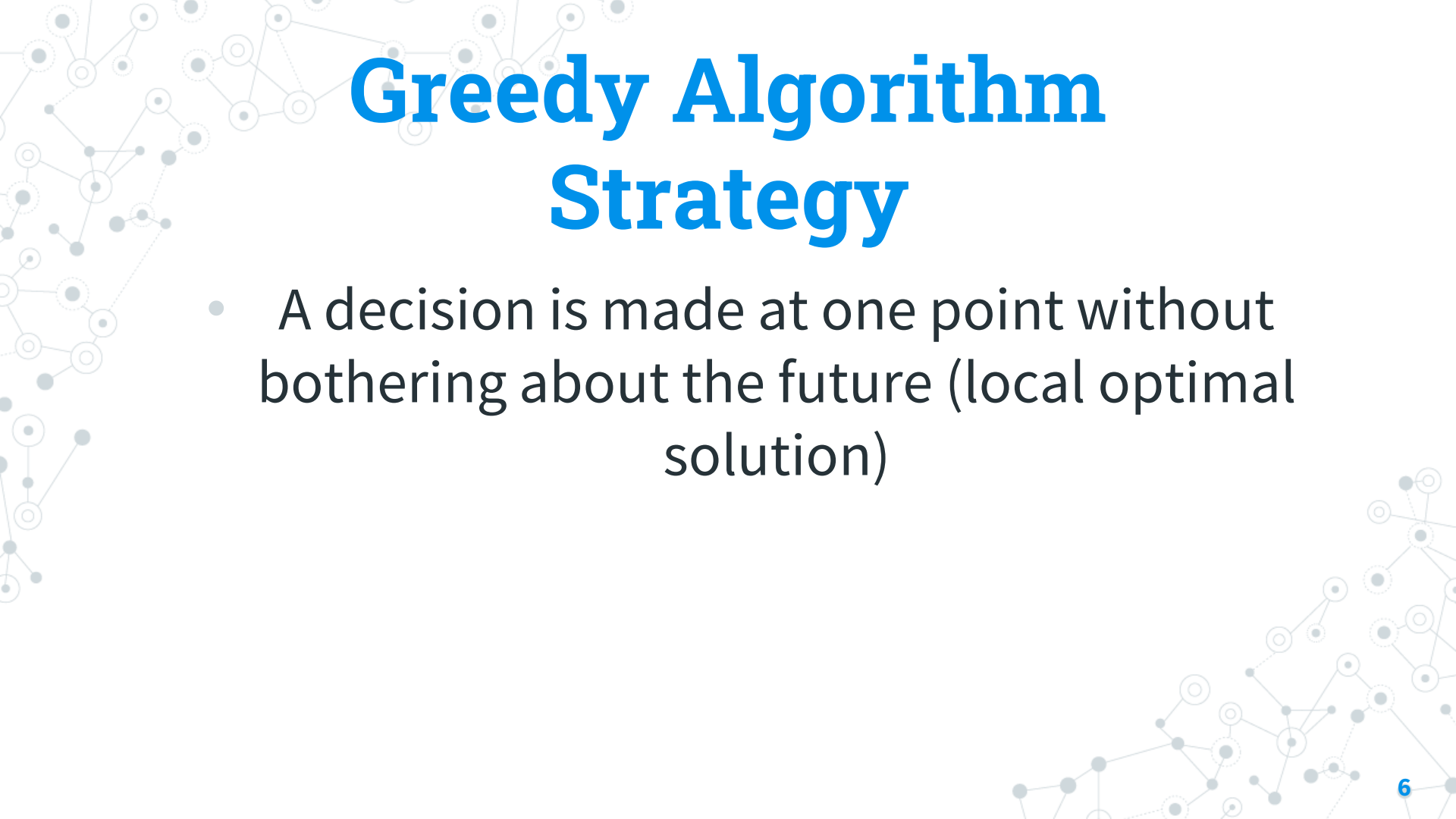
Algorithm Design Techniques

- * Greedy Algorithm
- * Divide and Conquer
- * Brute Force
- * Branch and Bound
- * Randomized Algorithm
- * Dynamic Programming
- * Genetic Algorithm



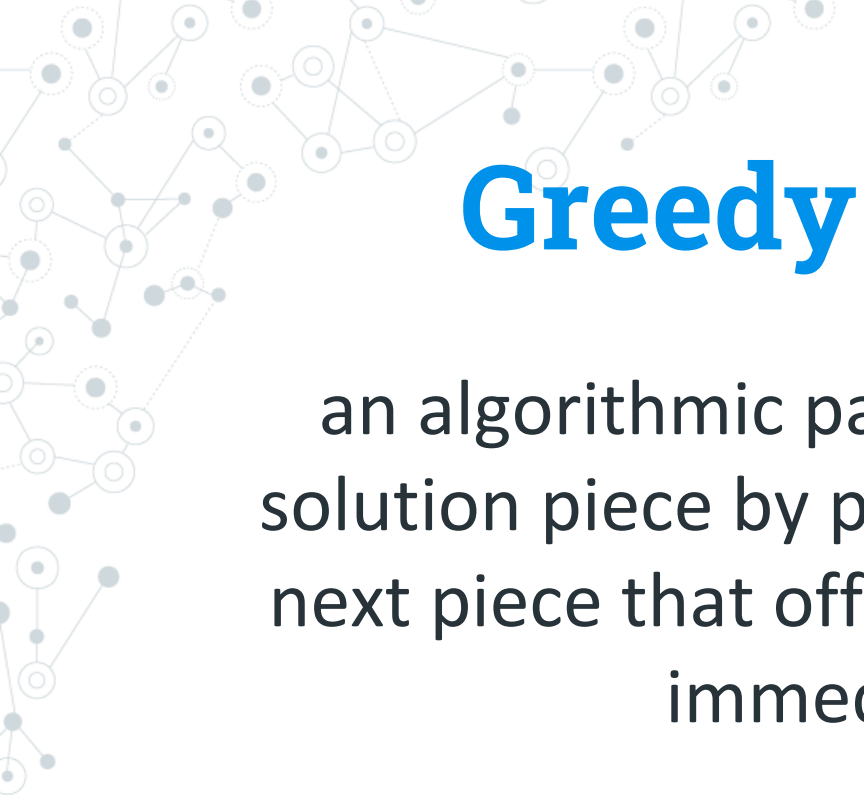
Greedy Algorithm

Chooses the best immediate/short-term solution rather than a long term solution.



Greedy Algorithm Strategy

- A decision is made at one point without bothering about the future (local optimal solution)



Greedy Algorithm

an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit.

Greedy Algorithm

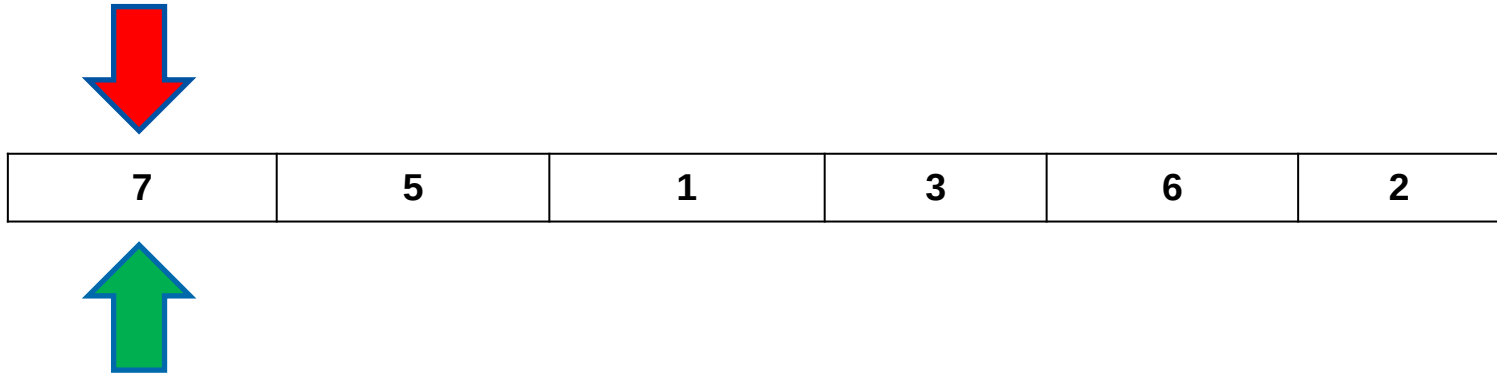
Example : Selection Sort

7	5	1	3	6	2
---	---	---	---	---	---

Sort from smallest to largest by picking the minimum value.

Greedy Algorithm

Example : Selection Sort

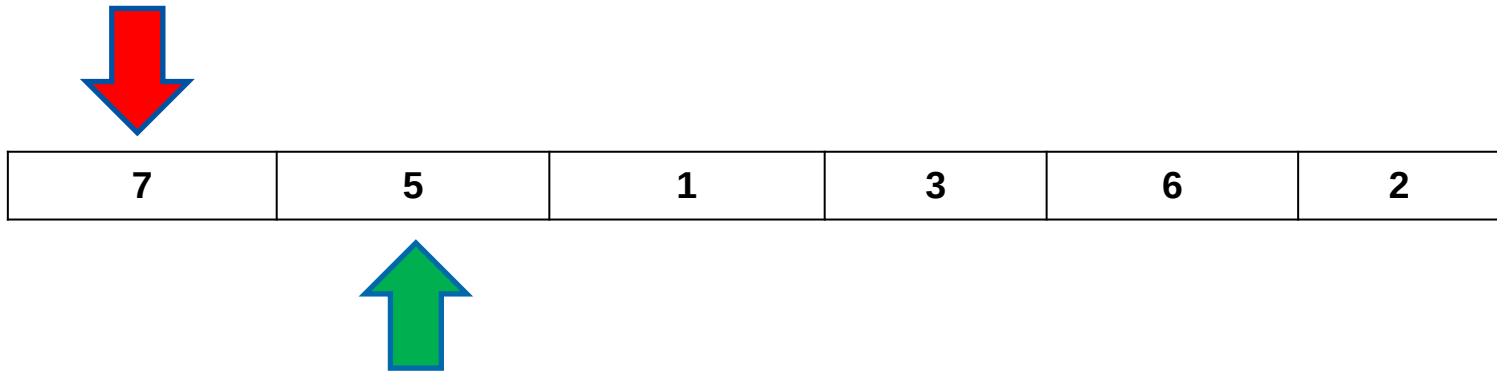


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



7	5	1	3	6	2
---	---	---	---	---	---

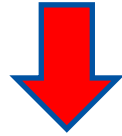


Always set the **current minimum** as the first element of the unsorted array,
every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



7	5	1	3	6	2
---	---	---	---	---	---



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



7	5	1	3	6	2
---	---	---	---	---	---



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



7	5	1	3	6	2
---	---	---	---	---	---



Always set the **current minimum** as the first element of the unsorted array,
every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



7	5	1	3	6	2
---	---	---	---	---	---



Always set the **current minimum** as the first element of the unsorted array,
every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



7	5	1	3	6	2
---	---	---	---	---	---

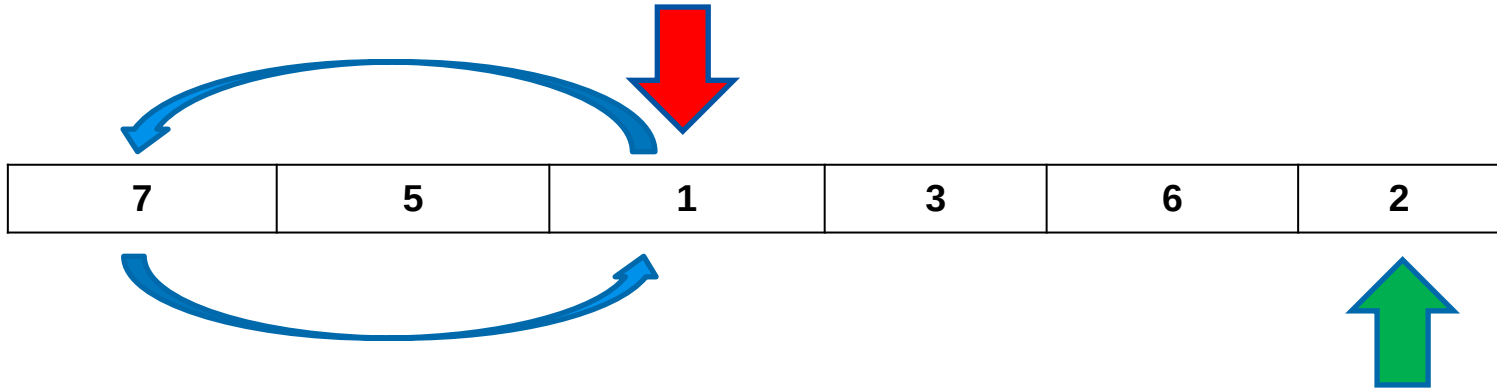


Always set the **current minimum** as the first element of the unsorted array,
every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

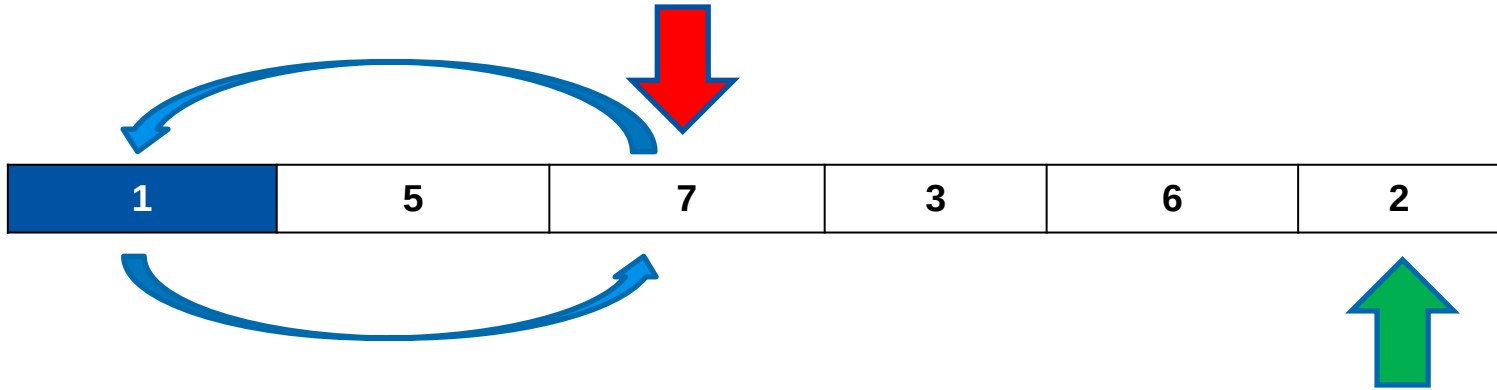


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

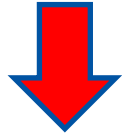


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



1	5	7	3	6	2
---	---	---	---	---	---

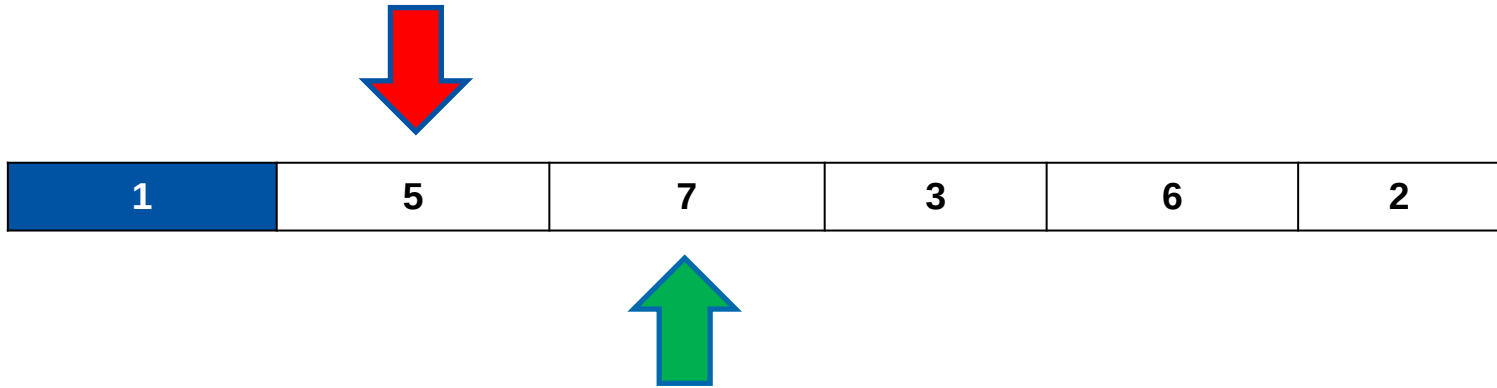


Always set the **current minimum** as the first element of the unsorted array,
every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

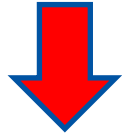


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



1	5	7	3	6	2
---	---	---	---	---	---

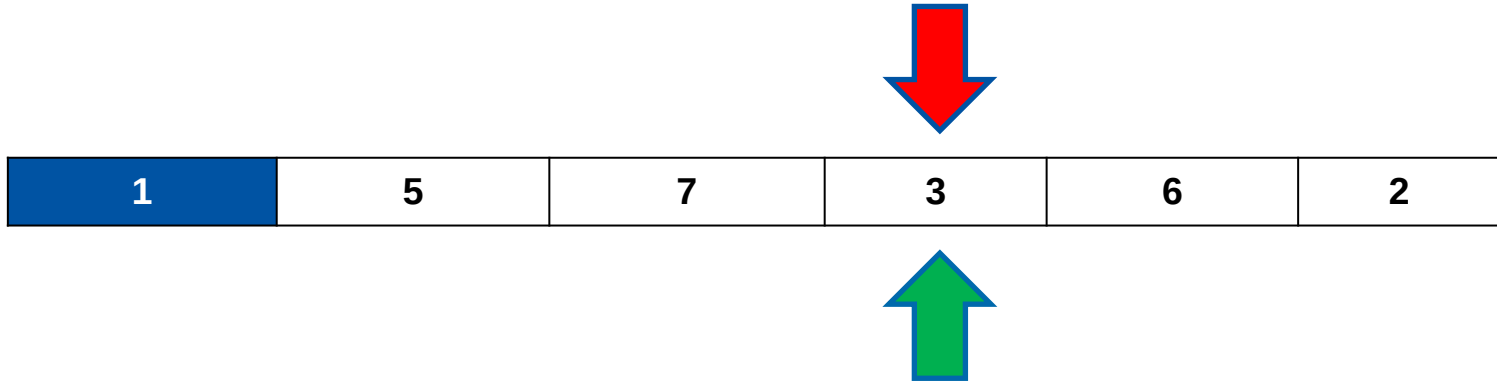


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

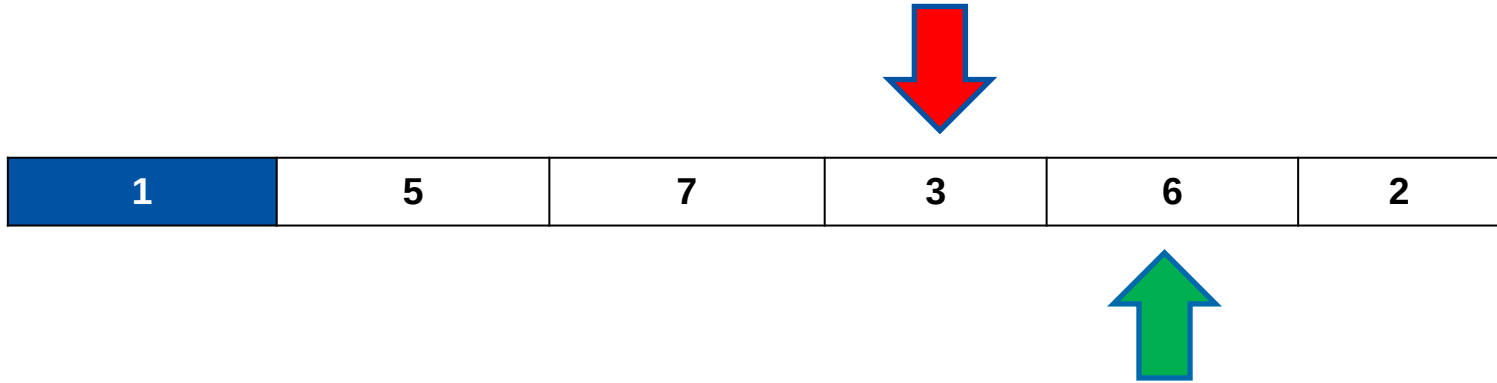


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

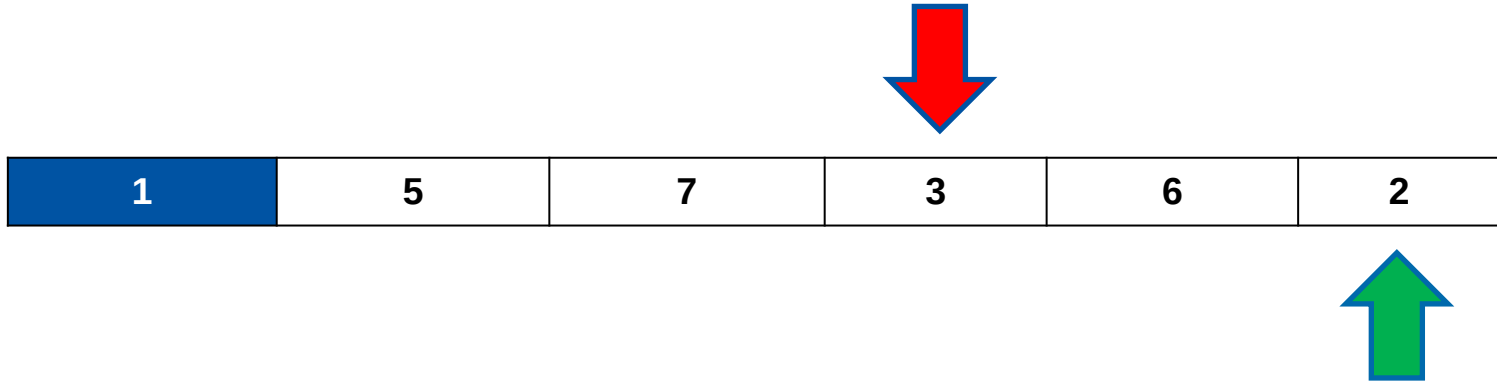


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



The diagram shows a horizontal array of six cells. The first cell contains the number 1 and is highlighted in dark blue. The other five cells contain the numbers 5, 7, 3, 6, and 2 in order from left to right. A large red arrow points down to the last cell (containing 2), and a large green arrow points up to the same cell.

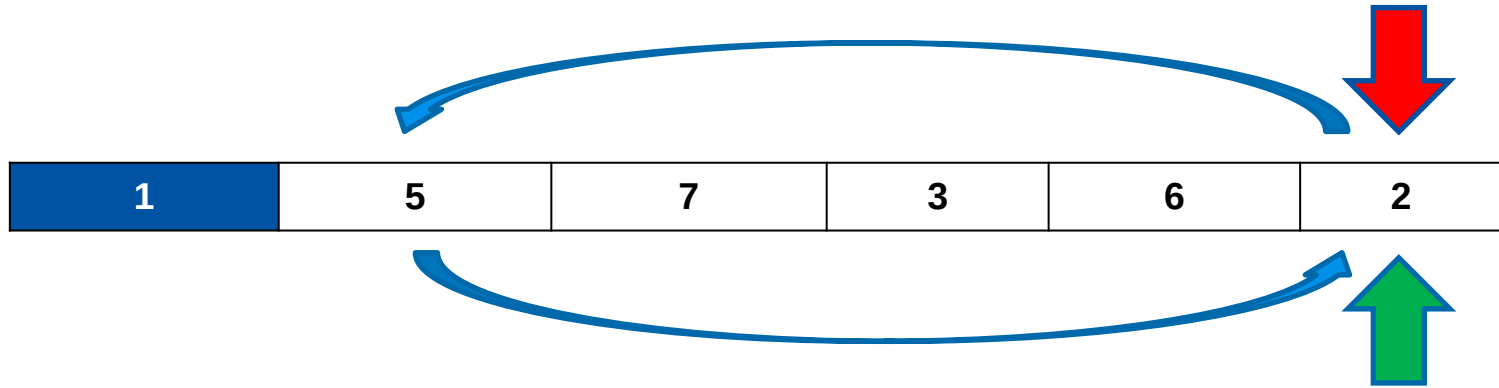
1	5	7	3	6	2
---	---	---	---	---	---

Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

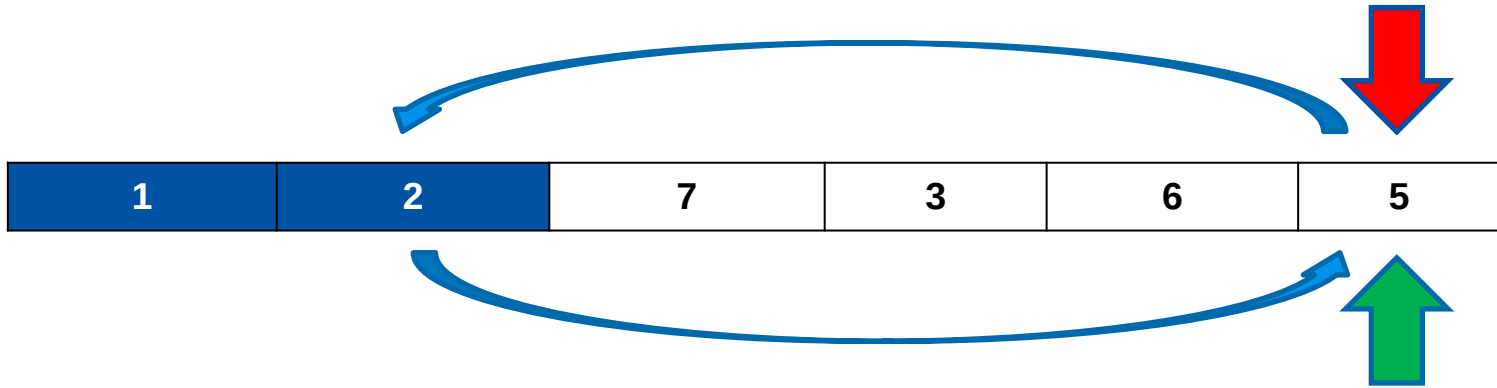


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

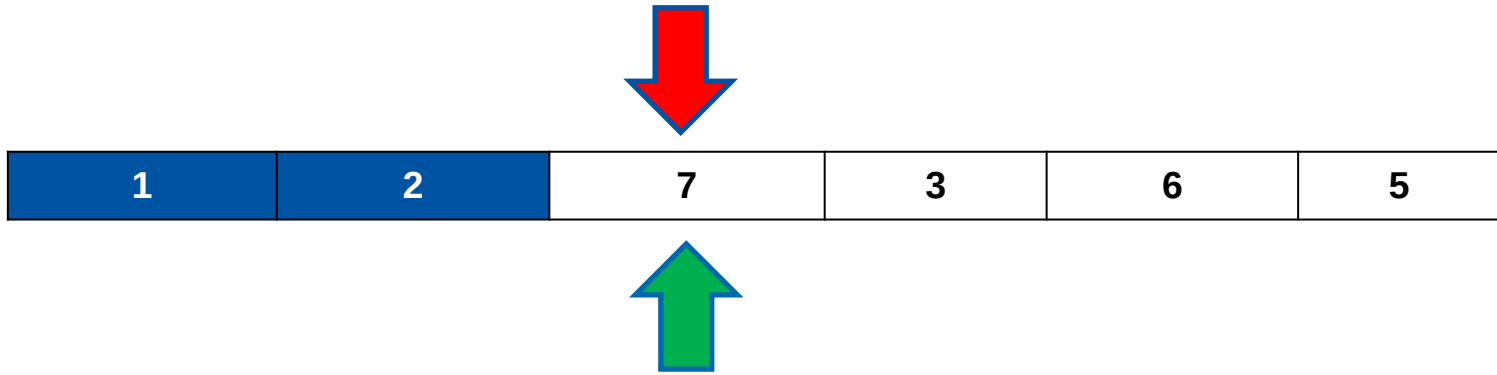


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

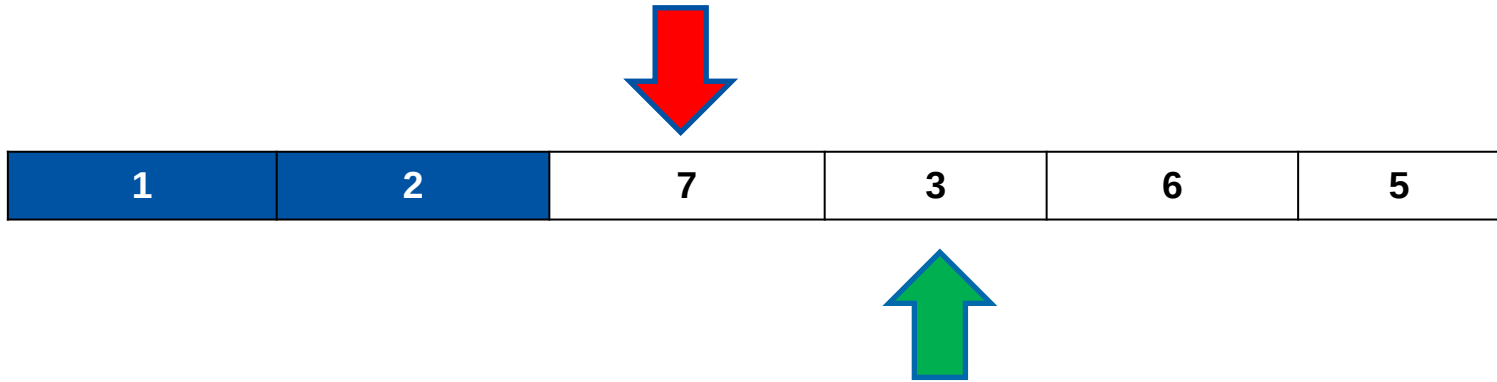


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



1	2	7	3	6	5
---	---	---	---	---	---



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



1	2	7	3	6	5
---	---	---	---	---	---



Always set the **current minimum** as the first element of the unsorted array,
every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



1	2	7	3	6	5
---	---	---	---	---	---

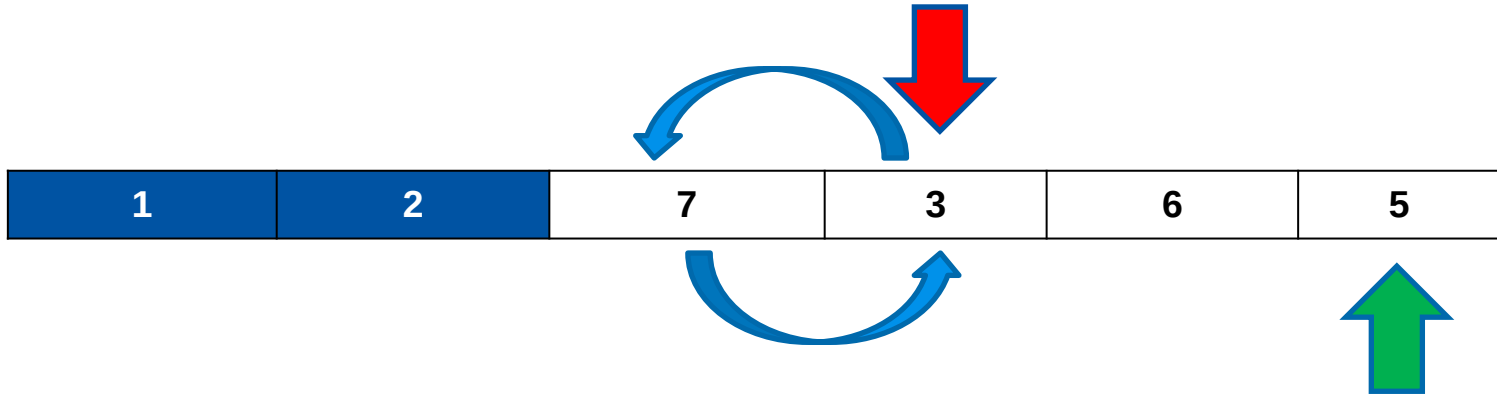


Always set the **current minimum** as the first element of the unsorted array,
every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

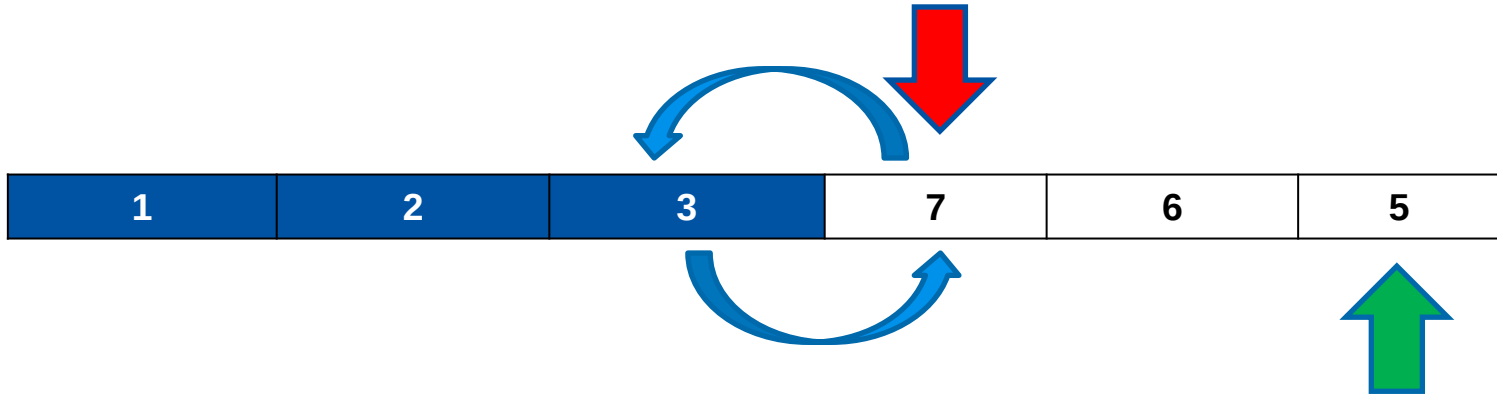


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

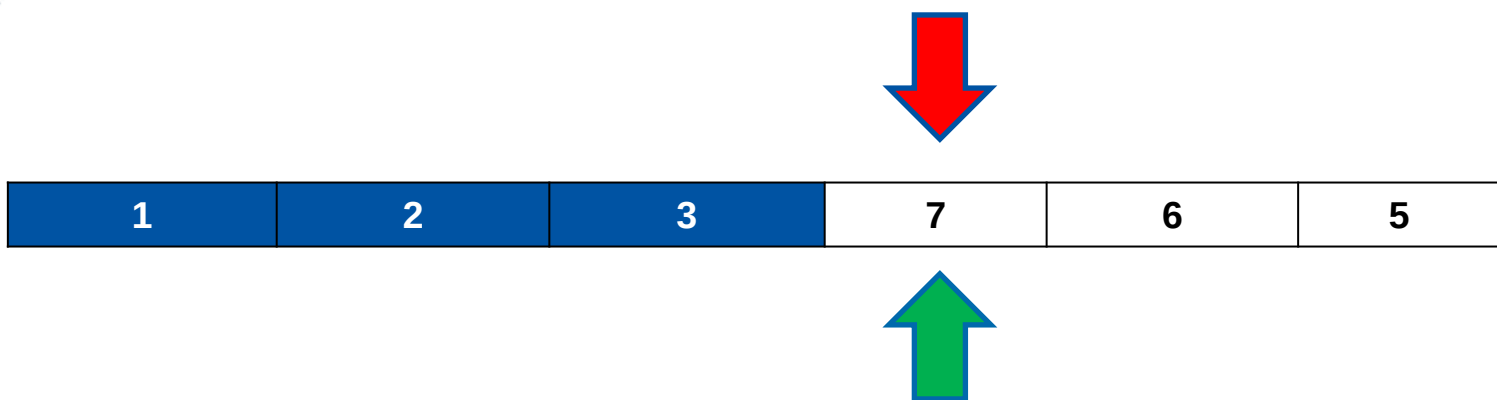


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



1	2	3	7	6	5
---	---	---	---	---	---

Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



1	2	3	7	6	5
---	---	---	---	---	---

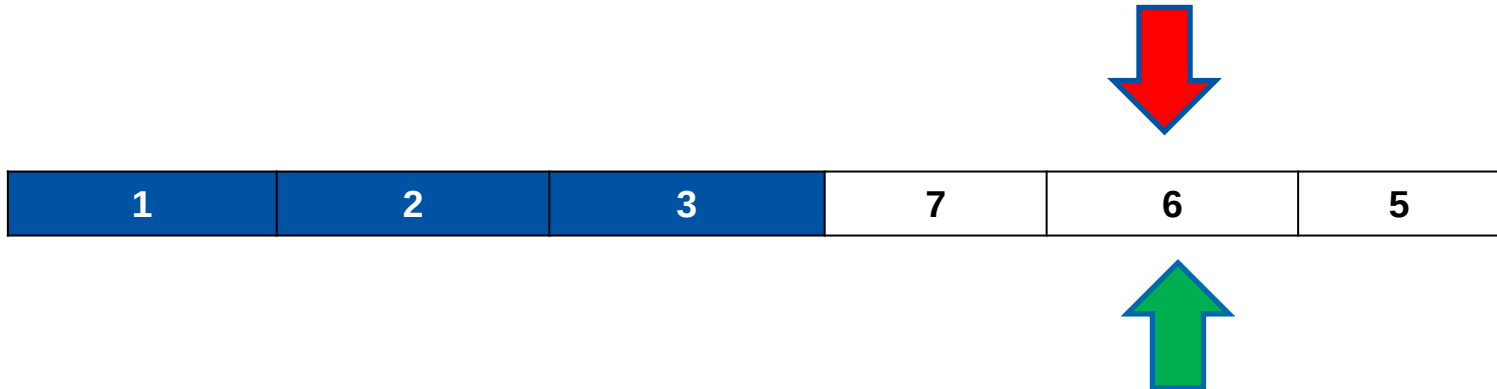


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



The diagram shows an array of six numbers: 1, 2, 3, 7, 6, 5. The first three numbers (1, 2, 3) are in dark blue boxes, indicating they are part of the sorted array. The last three numbers (7, 6, 5) are in white boxes with black borders, indicating they are part of the unsorted array. A large red arrow points down to the number 6, and a large green arrow points up to the number 6, indicating it is the current item being compared with the current minimum (3).

1	2	3	7	6	5
---	---	---	---	---	---

Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



1	2	3	7	6	5
---	---	---	---	---	---



Always set the **current minimum** as the first element of the unsorted array,
every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



The diagram shows an array of six numbers: 1, 2, 3, 7, 6, 5. The first three numbers (1, 2, 3) are in dark blue boxes, indicating they are part of the sorted array. The last three numbers (7, 6, 5) are in white boxes with black borders, indicating they are part of the unsorted array. A large red arrow points down to the unsorted section, and a large green arrow points up to the unsorted section.

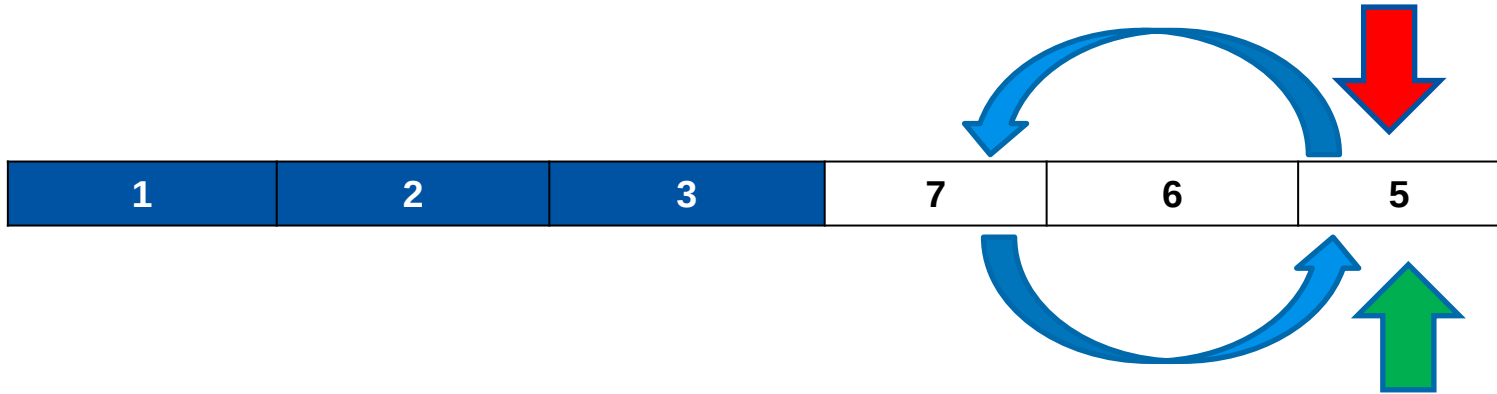
1	2	3	7	6	5
---	---	---	---	---	---

Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

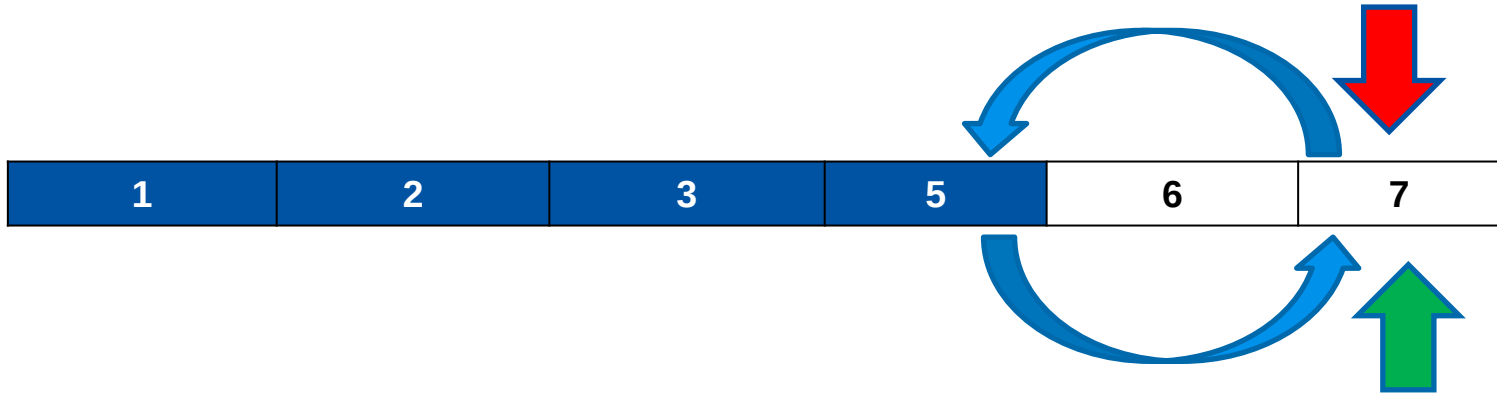


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend: Sorted Array Current minimum Current item

Greedy Algorithm

Example : Selection Sort

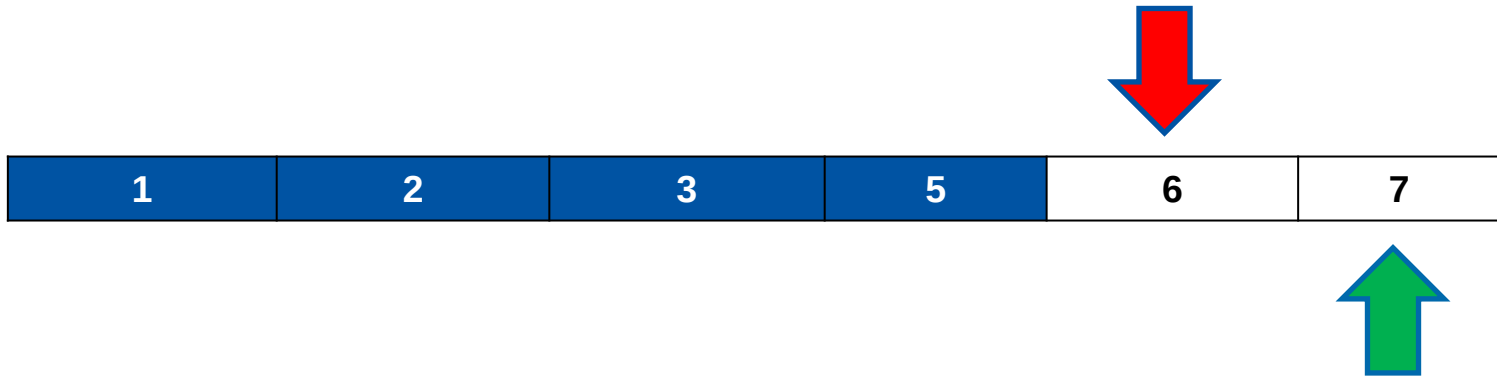


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort

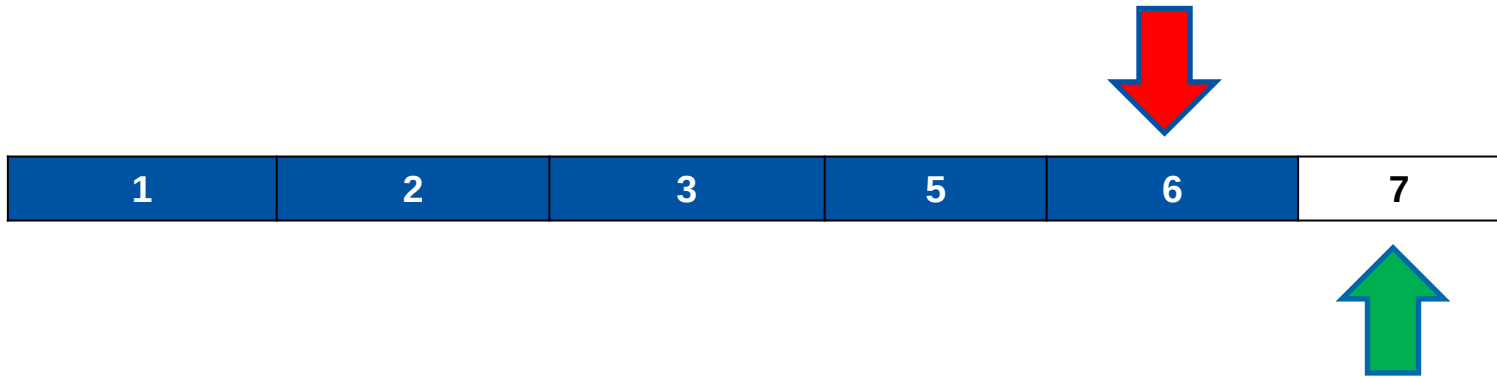


Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

Greedy Algorithm

Example : Selection Sort



Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item

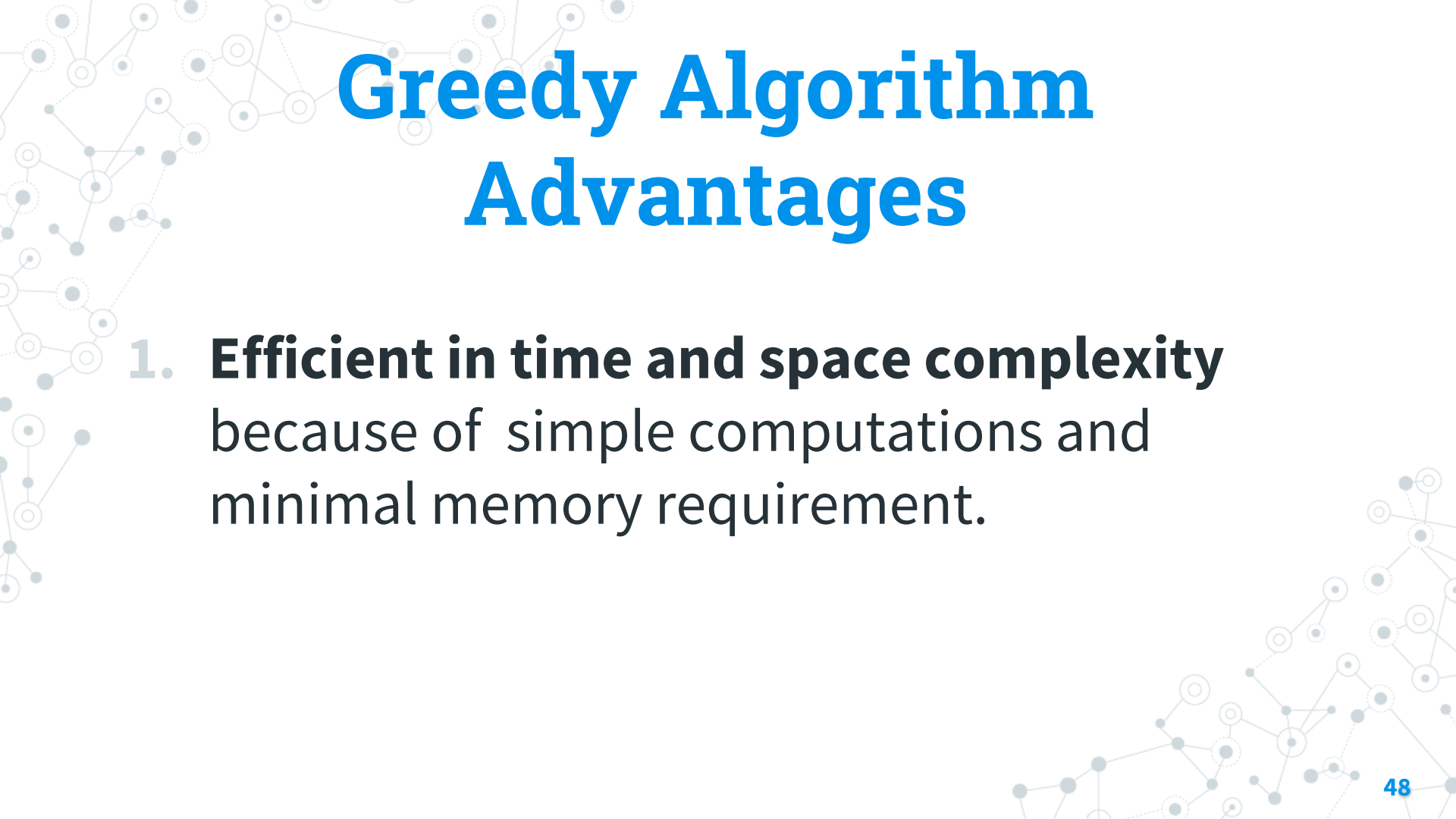
Greedy Algorithm

Example : Selection Sort

1	2	3	5	6	7
---	---	---	---	---	---

Always set the **current minimum** as the first element of the unsorted array, every start of iteration.

Legend:  Sorted Array  Current minimum  Current item



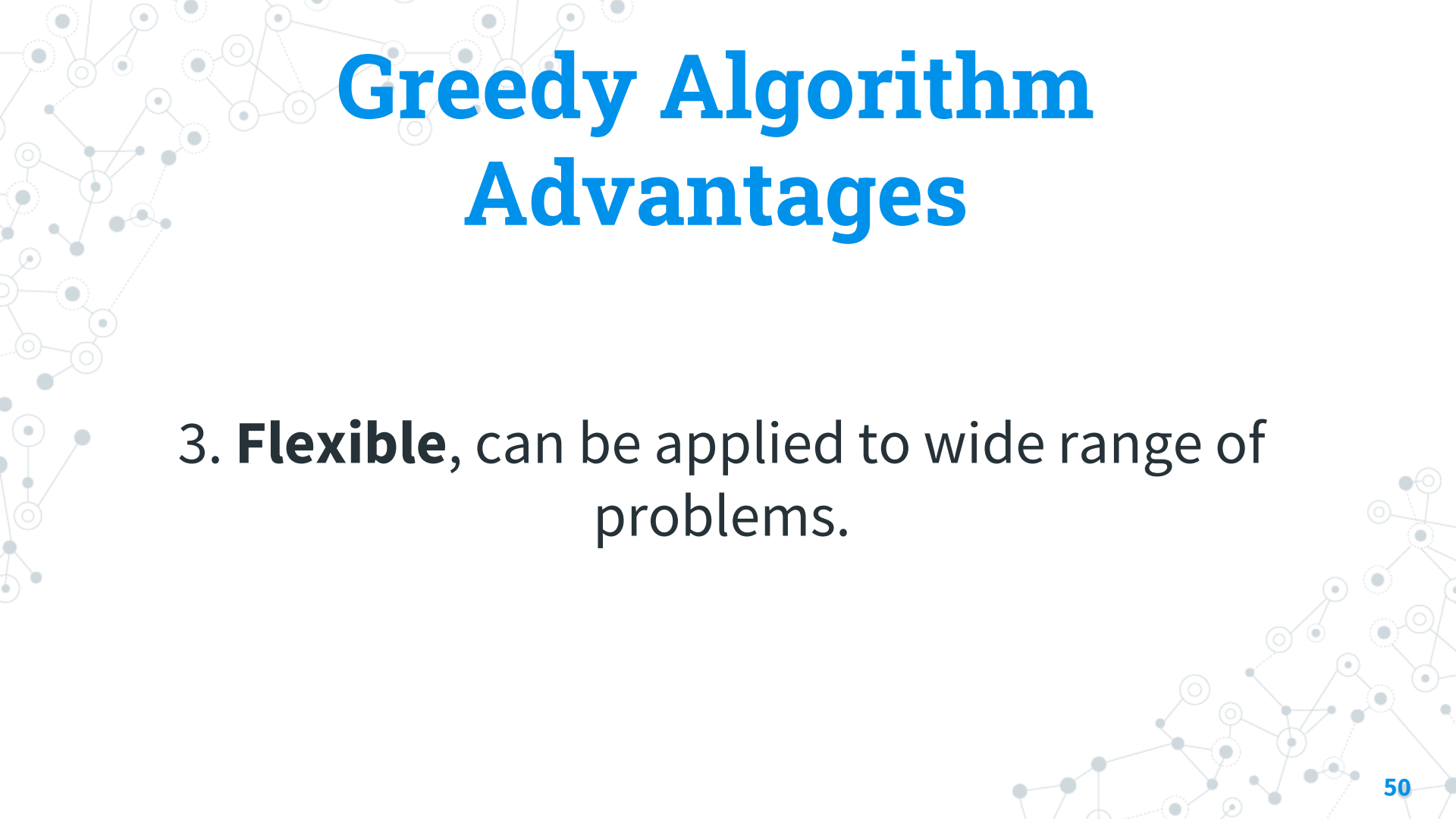
Greedy Algorithm Advantages

- 1. Efficient in time and space complexity**
because of simple computations and
minimal memory requirement.



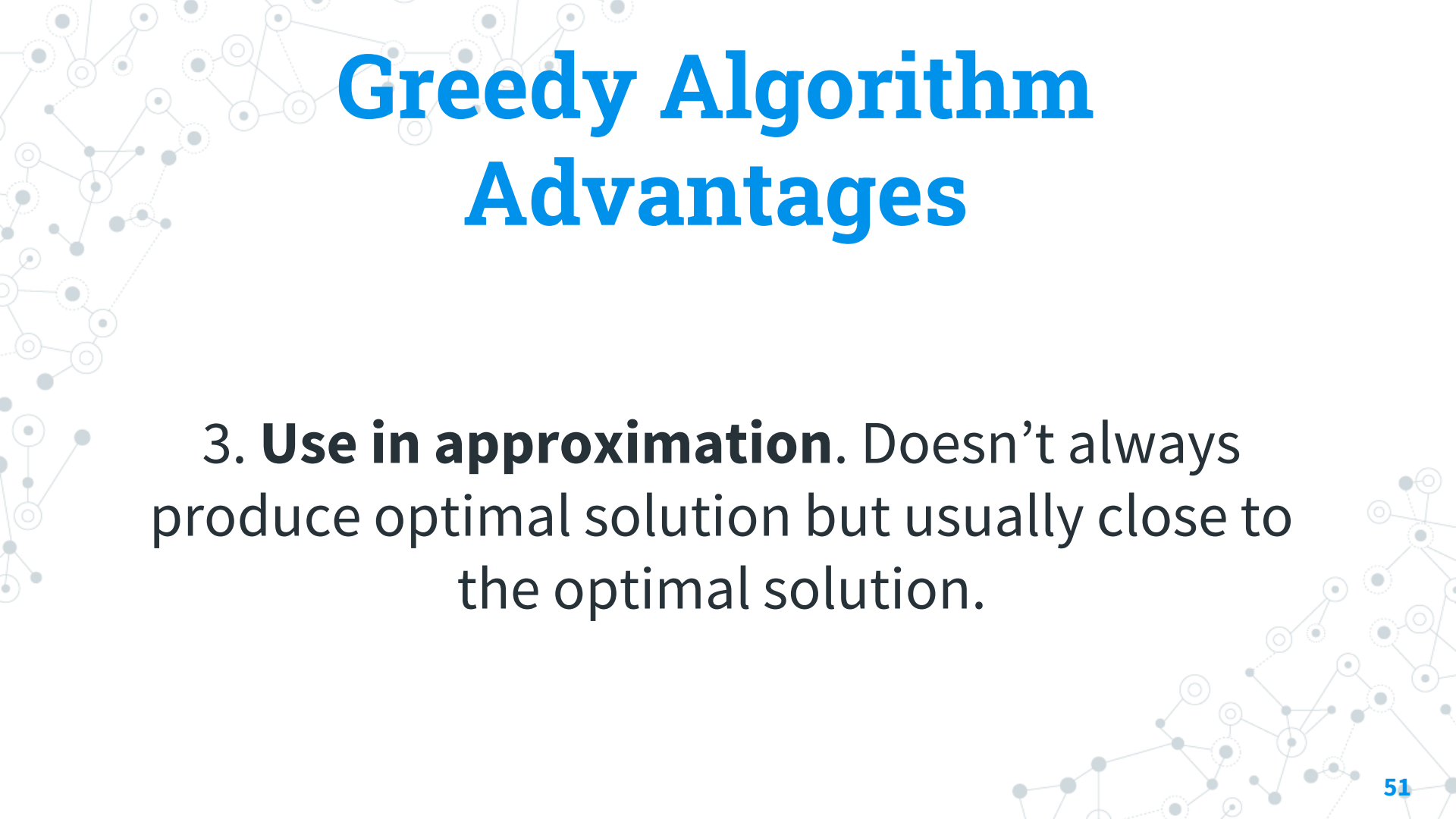
Greedy Algorithm Advantages

2. Simple and straightforward.



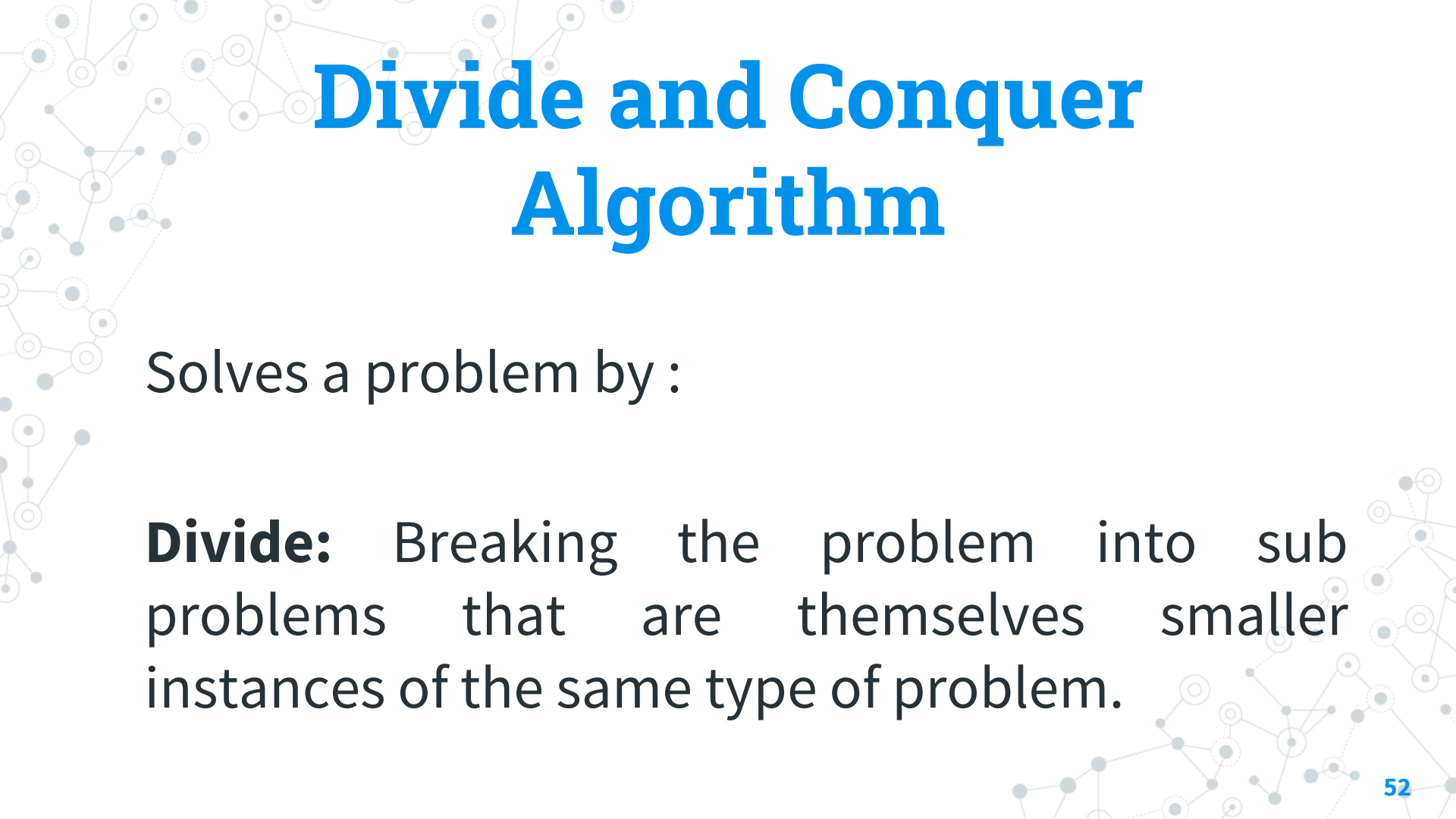
Greedy Algorithm Advantages

3. **Flexible**, can be applied to wide range of problems.



Greedy Algorithm Advantages

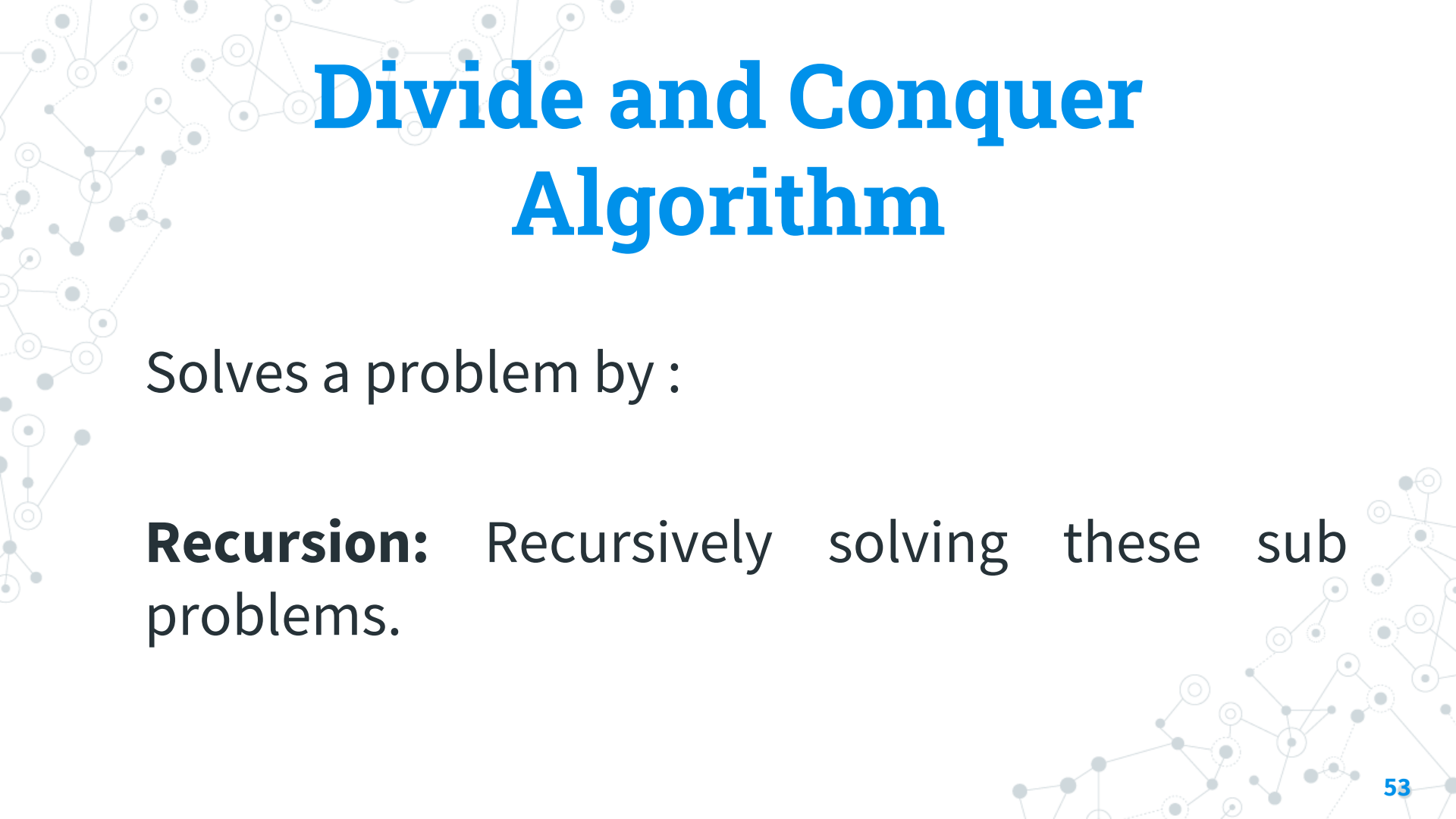
3. **Use in approximation.** Doesn't always produce optimal solution but usually close to the optimal solution.



Divide and Conquer Algorithm

Solves a problem by :

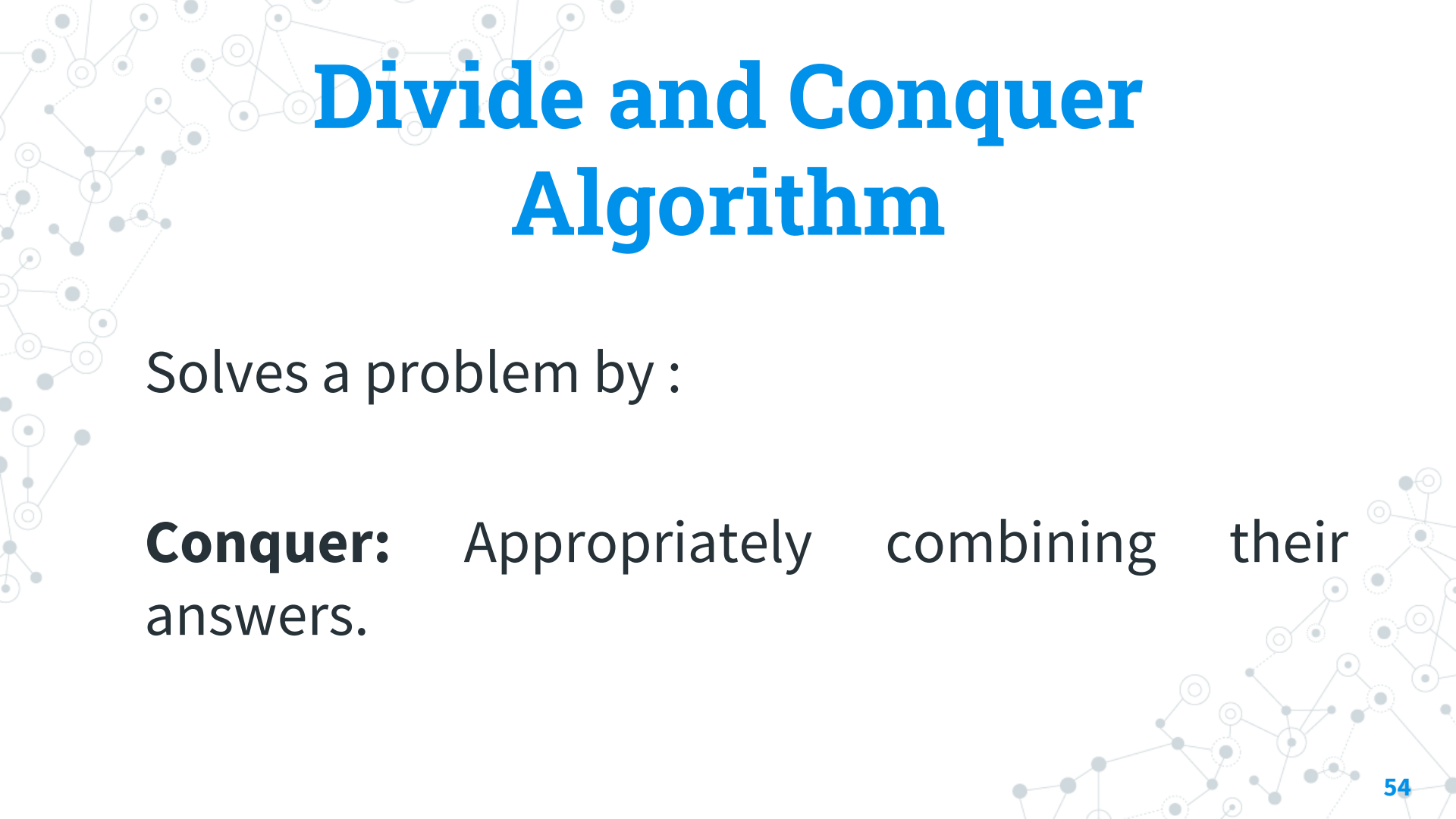
Divide: Breaking the problem into sub problems that are themselves smaller instances of the same type of problem.



Divide and Conquer Algorithm

Solves a problem by :

Recursion: Recursively solving these sub problems.



Divide and Conquer Algorithm

Solves a problem by :

Conquer: Appropriately combining their answers.

Divide and Conquer Algorithm

Example: Mergesort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

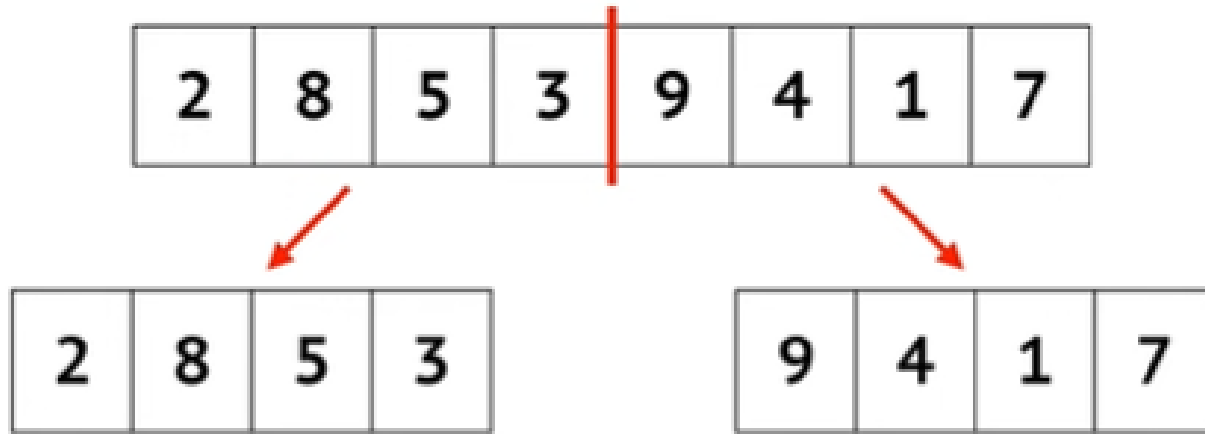
Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

Divide and Conquer Algorithm

Example: Merge sort



Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

2	8	5	3
---	---	---	---

9	4	1	7
---	---	---	---

Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

2	8	5	3
---	---	---	---

9	4	1	7
---	---	---	---

Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

2	8	5	3
---	---	---	---

9	4	1	7
---	---	---	---

2	8
---	---

5	3
---	---

9	4
---	---

1	7
---	---

Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

2	8	5	3
---	---	---	---

9	4	1	7
---	---	---	---

2	8
---	---

5	3
---	---

9	4
---	---

1	7
---	---

Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

2	8	5	3
---	---	---	---

9	4	1	7
---	---	---	---

2	8
---	---

5	3
---	---

9	4
---	---

1	7
---	---

Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

2	8	5	3
---	---	---	---

9	4	1	7
---	---	---	---

2	8
---	---

5	3
---	---

9	4
---	---

1	7
---	---

Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

2	8	5	3
---	---	---	---

9	4	1	7
---	---	---	---

2	8
---	---

5	3
---	---

9	4
---	---

1	7
---	---

2

8

5

3

9

4

1

7

Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

2	8	5	3
---	---	---	---

9	4	1	7
---	---	---	---

2	8
---	---

5	3
---	---

9	4
---	---

1	7
---	---

2

8

5

3

9

4

1

7

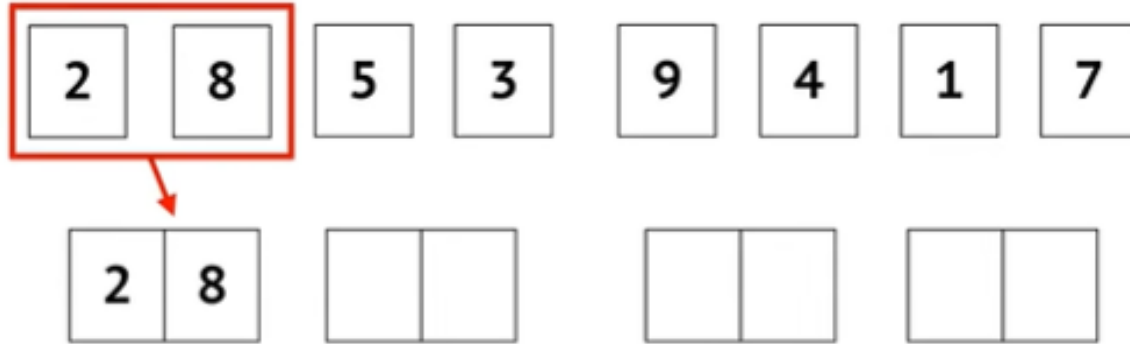
Divide and Conquer Algorithm

Example: Merge sort

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

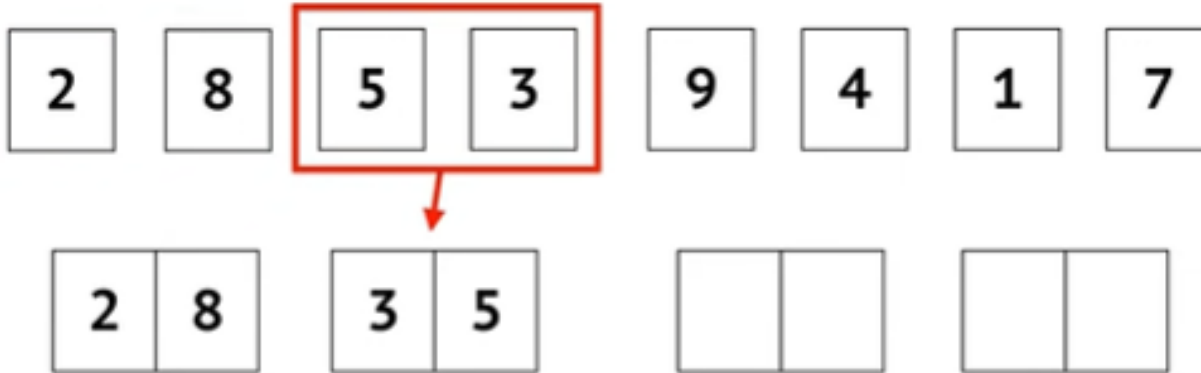
Divide and Conquer Algorithm

Example: Merge sort



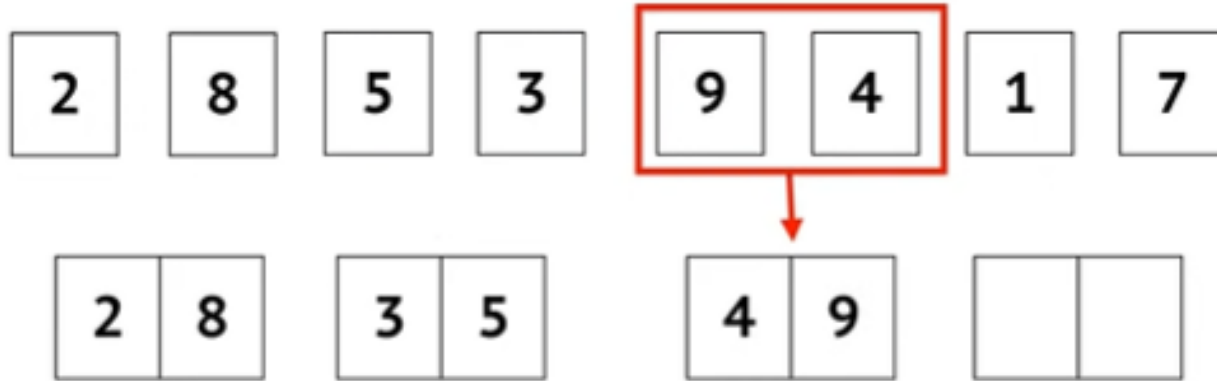
Divide and Conquer Algorithm

Example: Merge sort



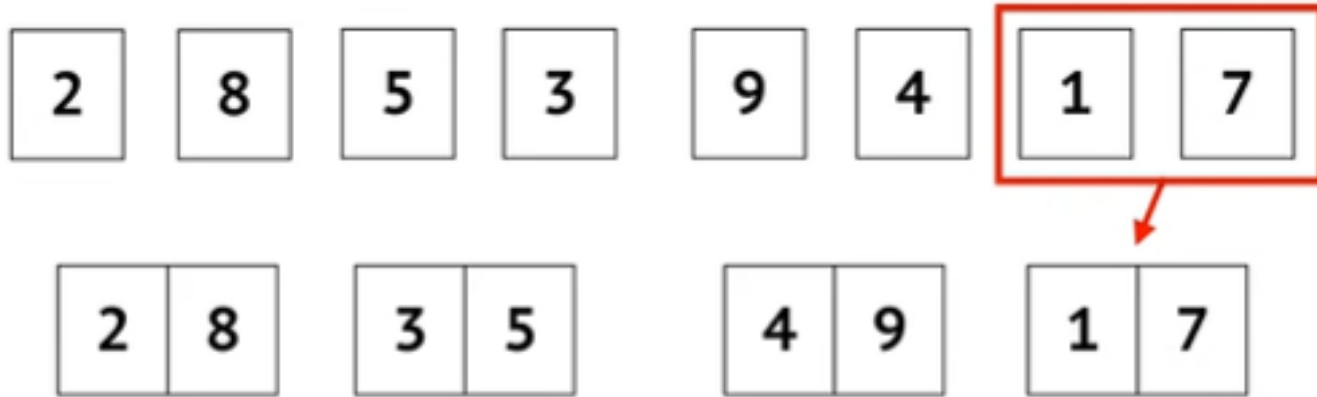
Divide and Conquer Algorithm

Example: Merge sort



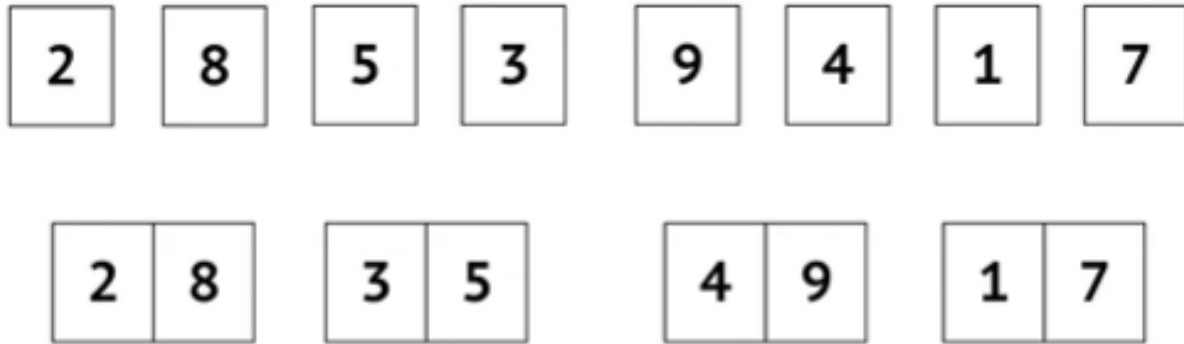
Divide and Conquer Algorithm

Example: Merge sort



Divide and Conquer Algorithm

Example: Merge sort



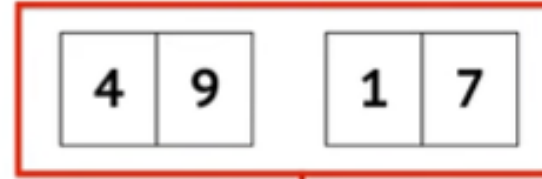
Divide and Conquer Algorithm

Example: Merge sort



Divide and Conquer Algorithm

Example: Merge sort



Divide and Conquer Algorithm

Example: Merge sort



Divide and Conquer Algorithm Advantages

Efficient : reduce time and resources required to solve complex problems.

Divide and Conquer Algorithm Advantages

Simplification: Dividing a complex problem into smaller can simply the problem-solving process.

Divide and Conquer Algorithm Advantages

Reusable: Solutions to subproblems can be reused in future problem-solving efforts.

Divide and Conquer Algorithm Advantages

Parallelism: Can be used in parallel computing environments

Divide and Conquer Algorithm

Disadvantages

Overhead: Dividing a problem into smaller subproblems and then combining the solutions can sometimes require additional overhead, such as communication and synchronization between the subproblems.

Divide and Conquer Algorithm

Disadvantages

Complexity: Dividing a problem into smaller subproblems can sometimes make the problem more complex, requiring additional effort to understand and solve each subproblem, and then combining the solutions.

Divide and Conquer Algorithm Disadvantages

Subproblem dependency: In some cases, the solutions to subproblems may depend on each other, making it difficult to solve them independently. In such cases, the divide and conquer technique may not be suitable.

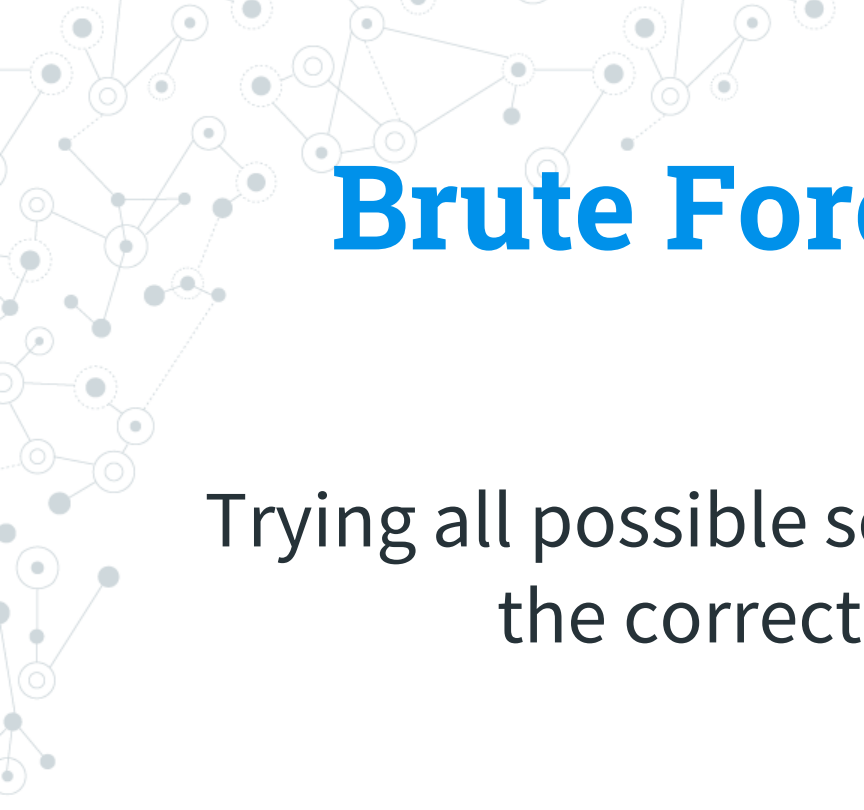
Divide and Conquer Algorithm

Disadvantages

Not always applicable: May not be suitable for all types of problems, especially those that do not have divisible nature.

Divide and Conquer Algorithm Disadvantages

Resource requirements: Dividing a problem into smaller subproblems and then solving each subproblem independently may require additional resources, such as memory and processing power.

A decorative network diagram in the top-left corner, consisting of a complex web of interconnected nodes and lines. The nodes are represented by small circles, some of which are highlighted with a double-circle outline. The lines are thin and gray, creating a mesh-like structure.

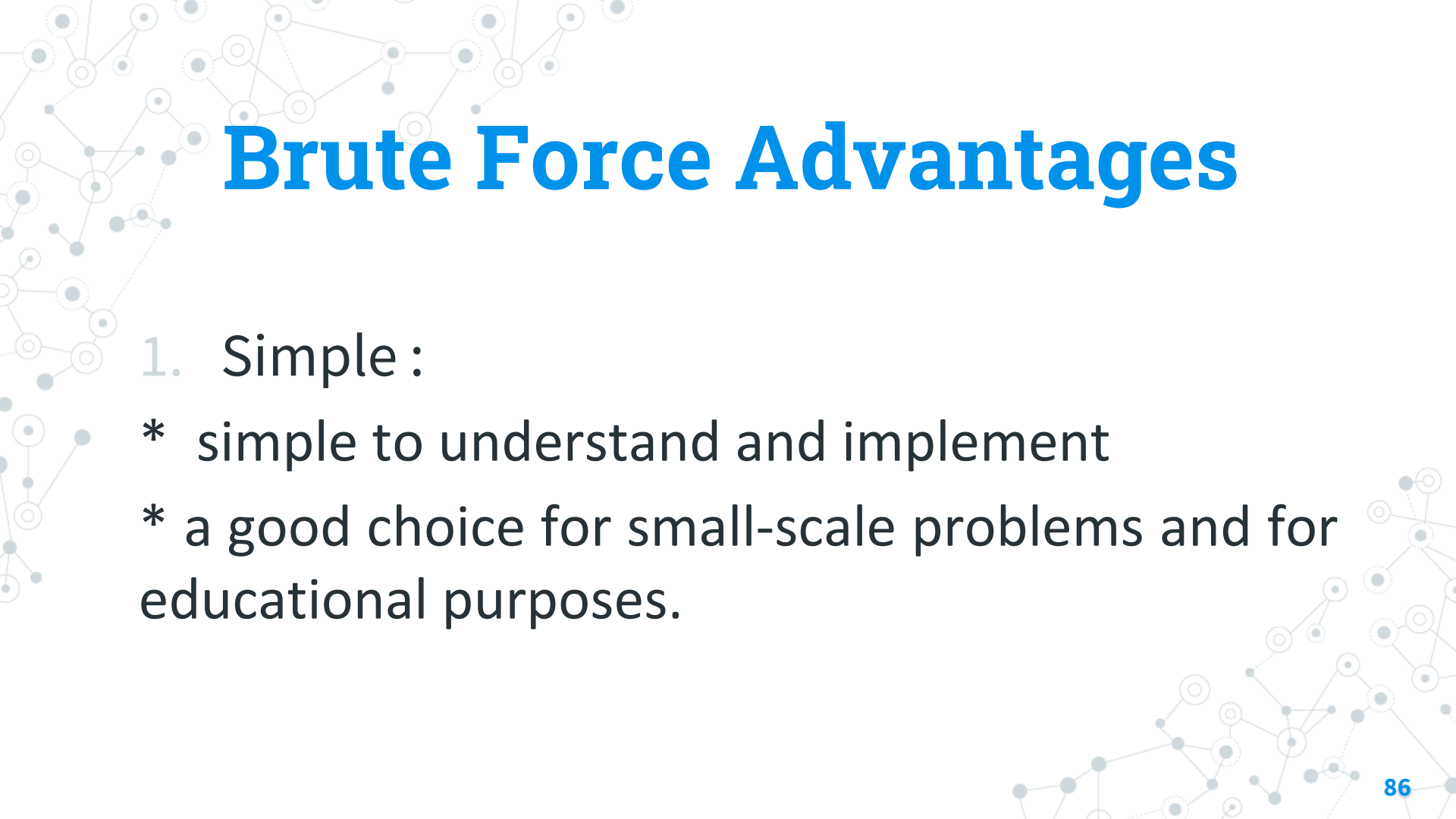
Brute Force Technique

Trying all possible solutions to a problem until the correct solution is found.

Brute Force Example

Cracking a 16-digit password.

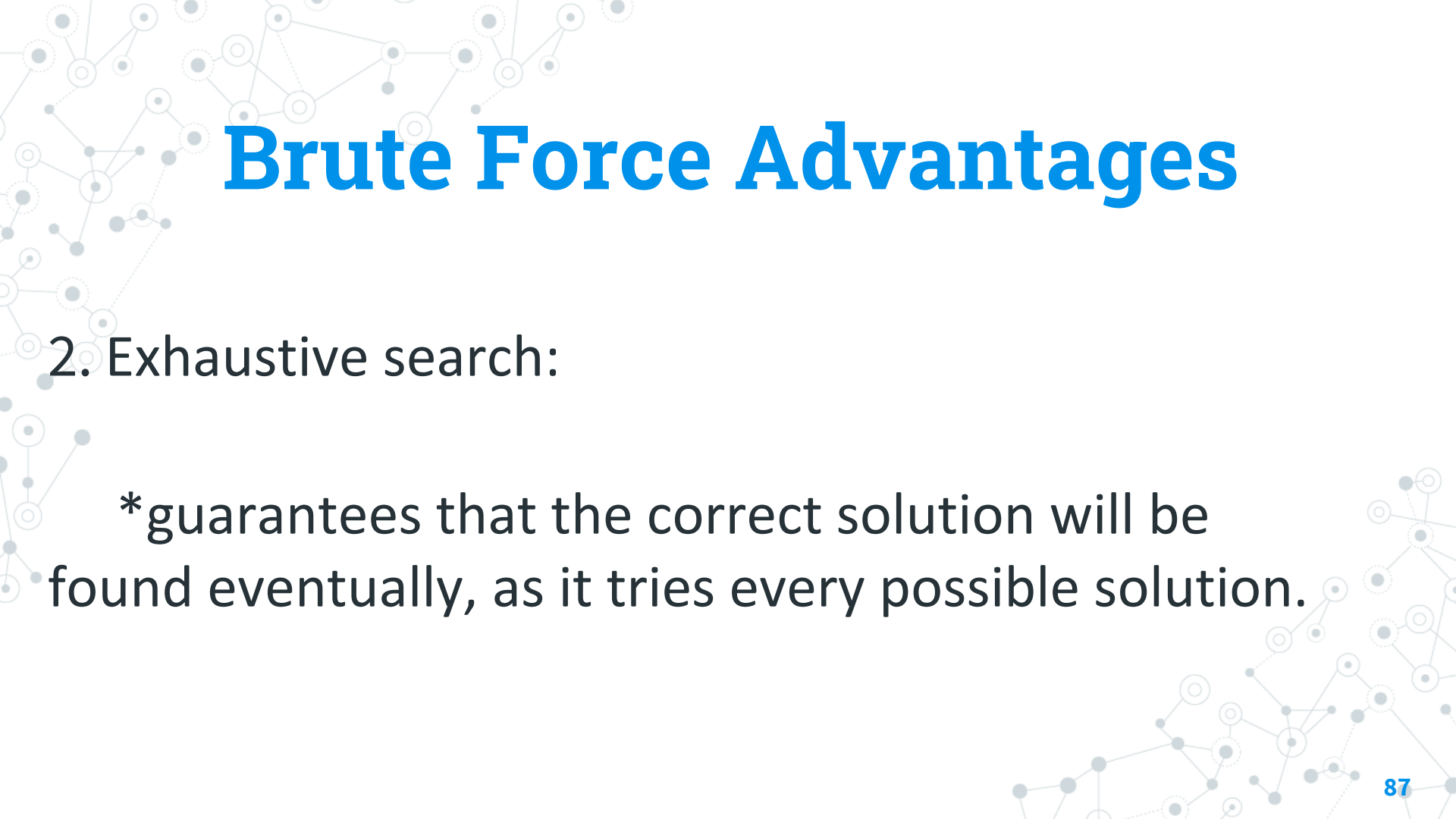
Character set	Number of Characters	Combinations	Time (years)
(a-z)	26	4.4×10^{22}	1.394×10^{15}
(a-z, A-Z)	52	2.9×10^{27}	9.1×10^{19}
(a-z, A-Z, 0-9)	62	4.8×10^{28}	1.5×10^{21}
(a-z, A-Z, 0-9, !@#\$%^&*...)	92	2.6×10^{31}	8.3×10^{23}



Brute Force Advantages

1. Simple :

- * simple to understand and implement
- * a good choice for small-scale problems and for educational purposes.



Brute Force Advantages

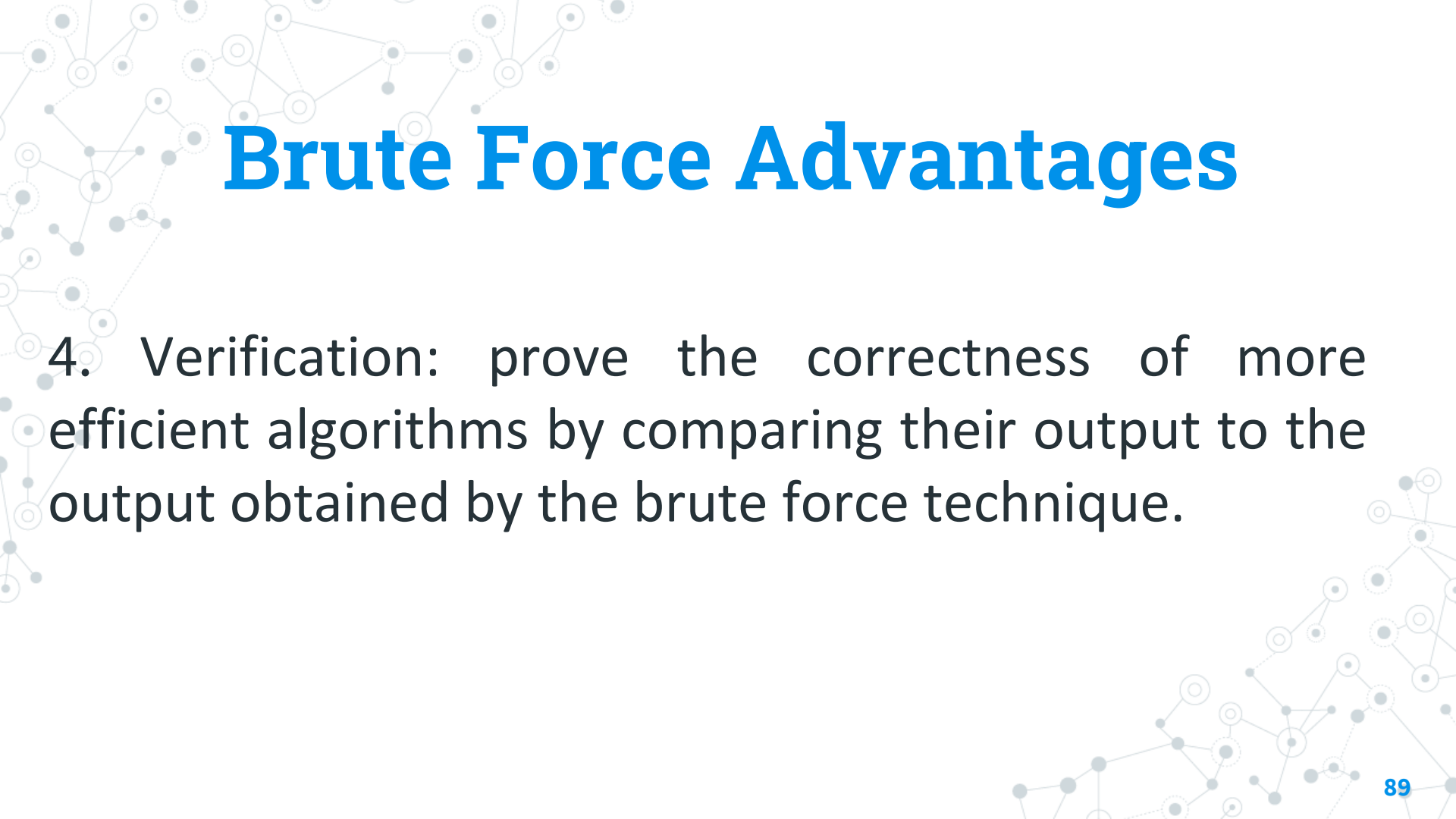
2. Exhaustive search:

*guarantees that the correct solution will be found eventually, as it tries every possible solution.

Brute Force Advantages

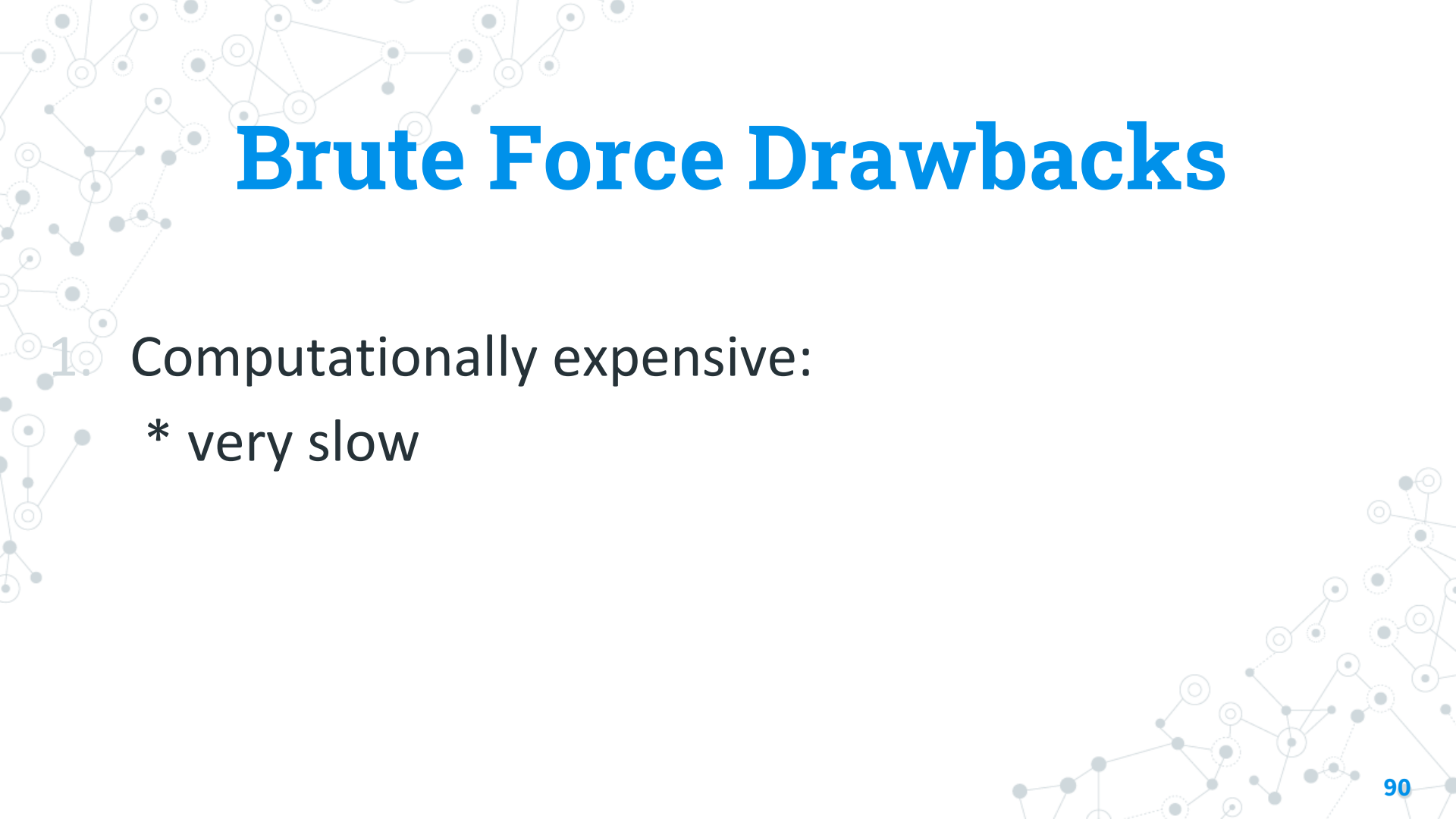
3. Versatility:

- * can be applied to a wide range of problems, regardless of their complexity or structure.



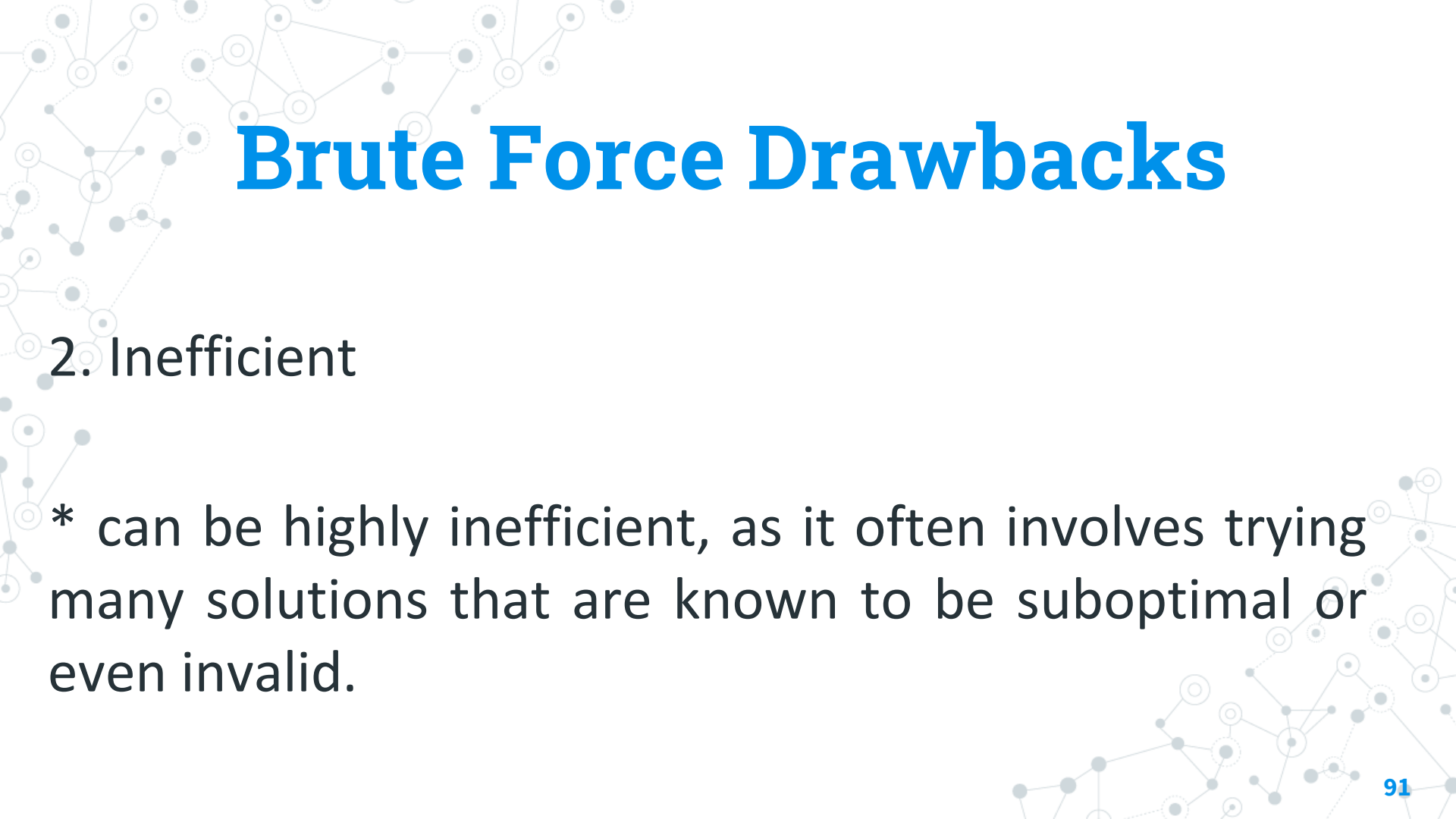
Brute Force Advantages

4. Verification: prove the correctness of more efficient algorithms by comparing their output to the output obtained by the brute force technique.



Brute Force Drawbacks

1. Computationally expensive:
 - * very slow



Brute Force Drawbacks

2. Inefficient

* can be highly inefficient, as it often involves trying many solutions that are known to be suboptimal or even invalid.

Brute Force Drawbacks

3. Limited Scalability

* Impractical or even impossible to use for problems with a very large search space / high degree of complexity.



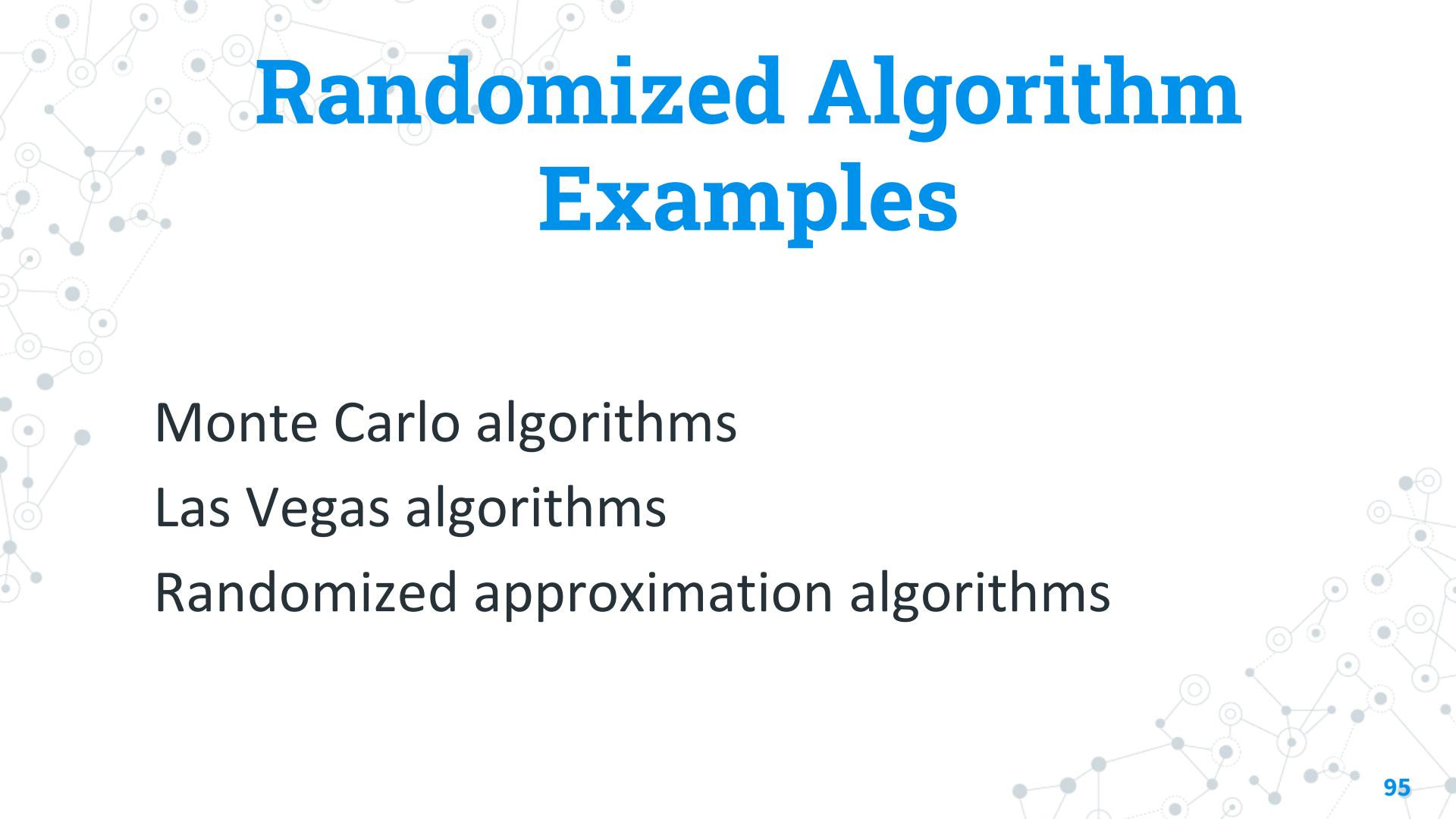
Branch and Bound

Improvised version of brute force. If a solution is worse than the current best solution, it backtracks and finds another solution.



Randomized Algorithm

uses randomization as an essential part of its design and operation.

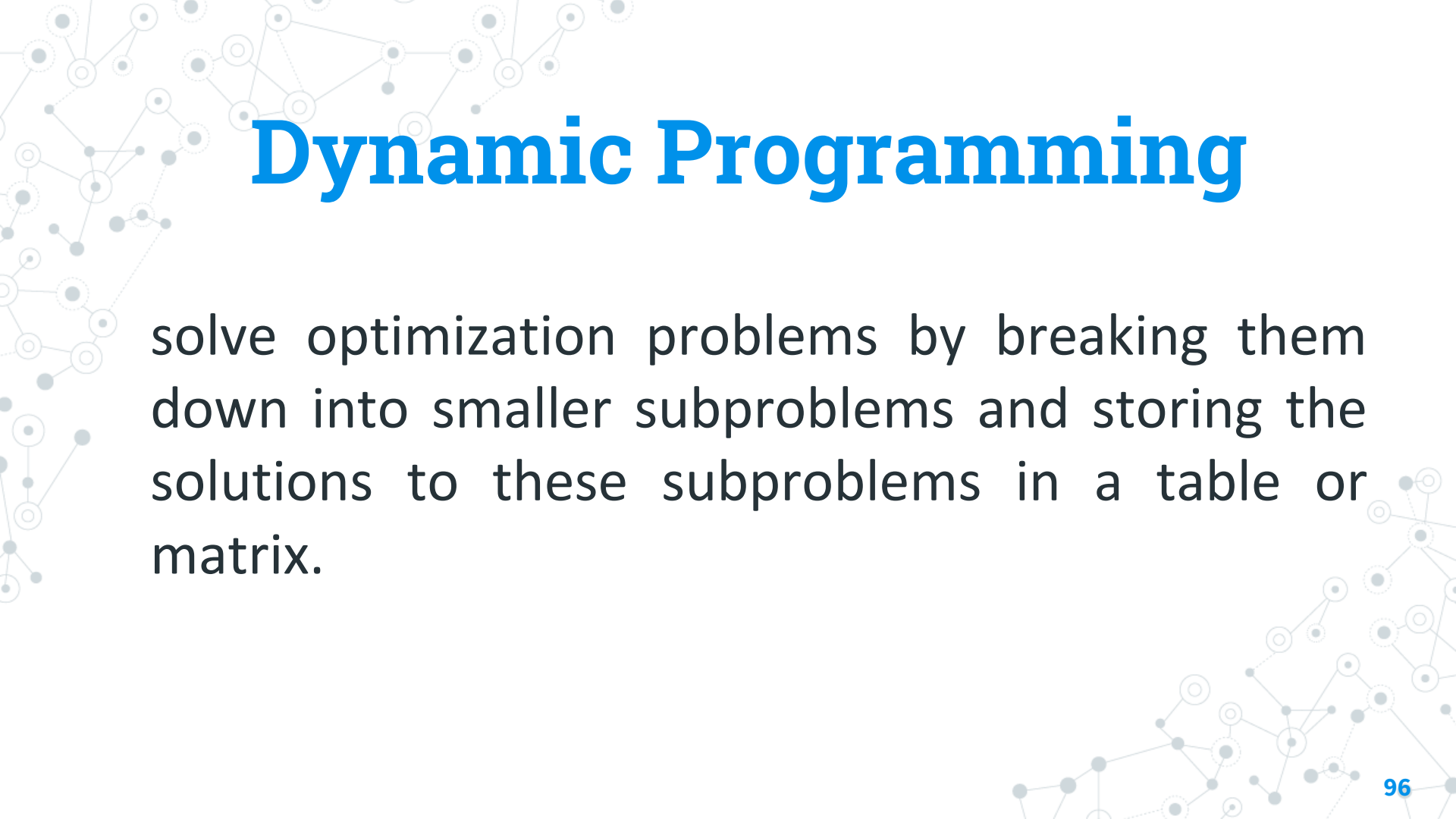


Randomized Algorithm Examples

Monte Carlo algorithms

Las Vegas algorithms

Randomized approximation algorithms



Dynamic Programming

solve optimization problems by breaking them down into smaller subproblems and storing the solutions to these subproblems in a table or matrix.

Dynamic Programming Problems

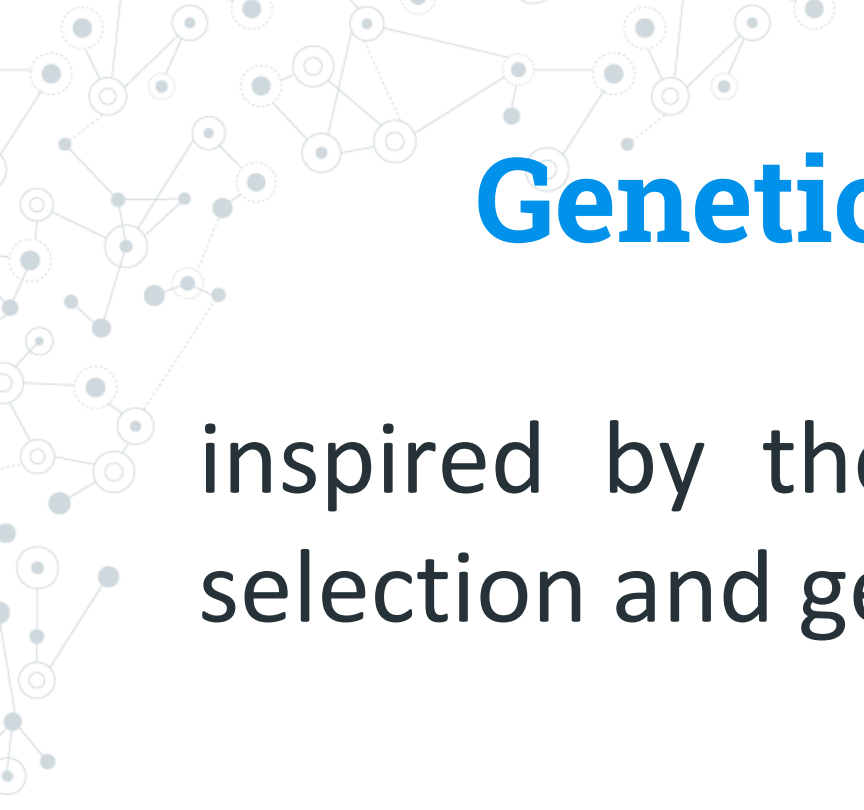
Fibonacci Sequence

Shortest path problem

Knapsack Problem

Longest increasing Subsequence

Matrix Chain Multiplication



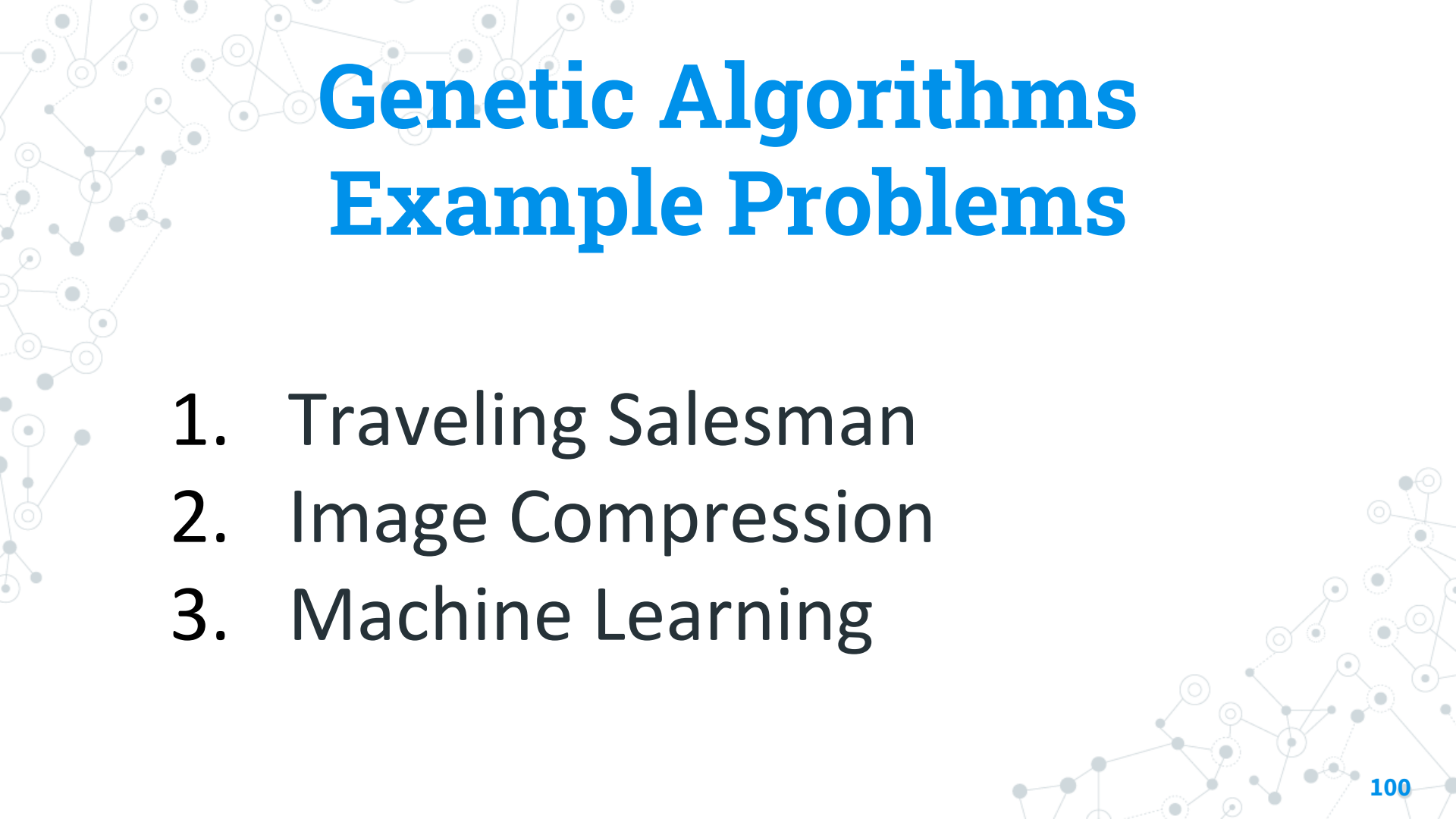
Genetic Algorithm

inspired by the process of natural selection and genetics.



Genetic Algorithm Operators

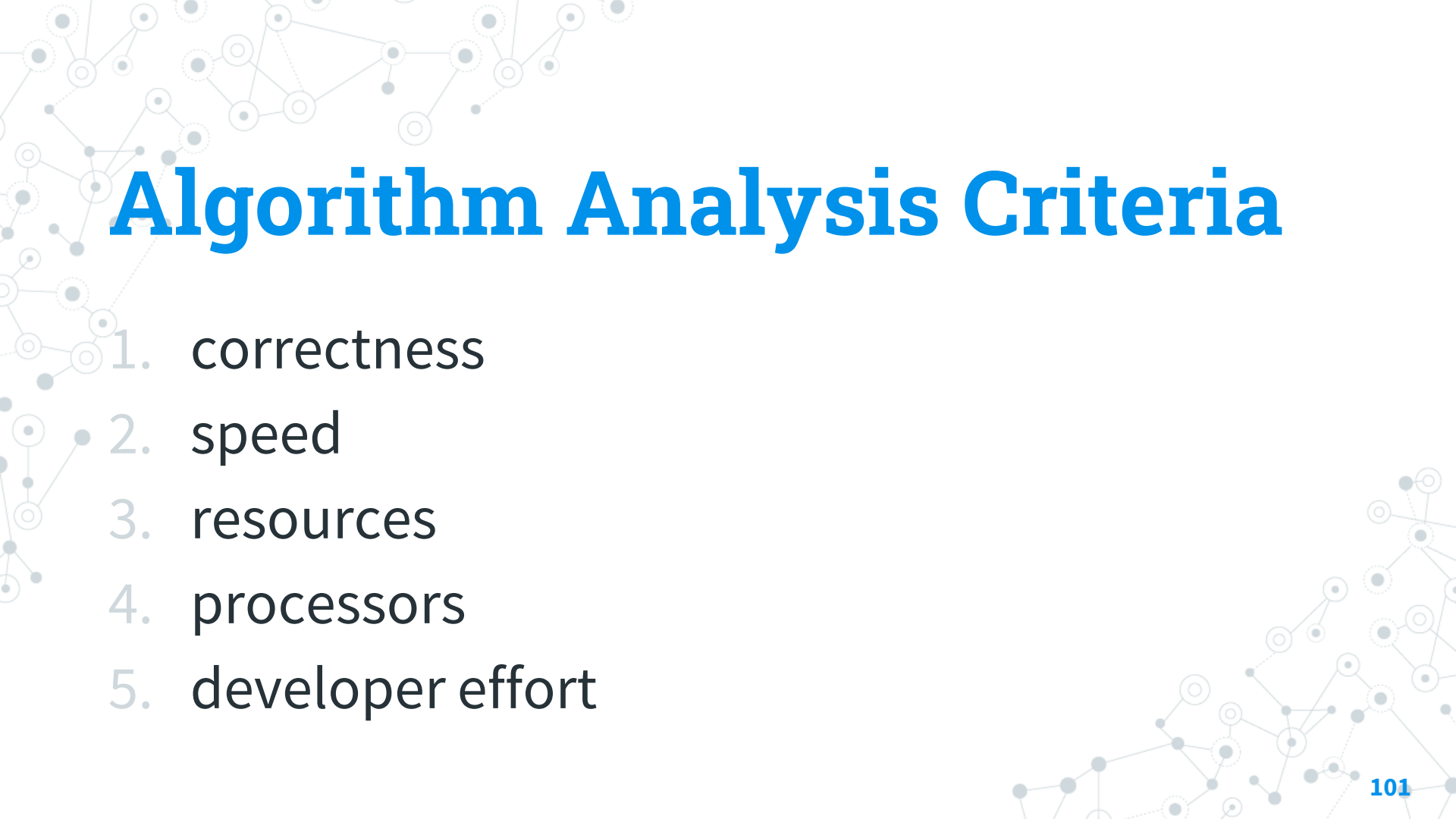
1. Selection
2. Crossover
3. Mutation



Genetic Algorithms

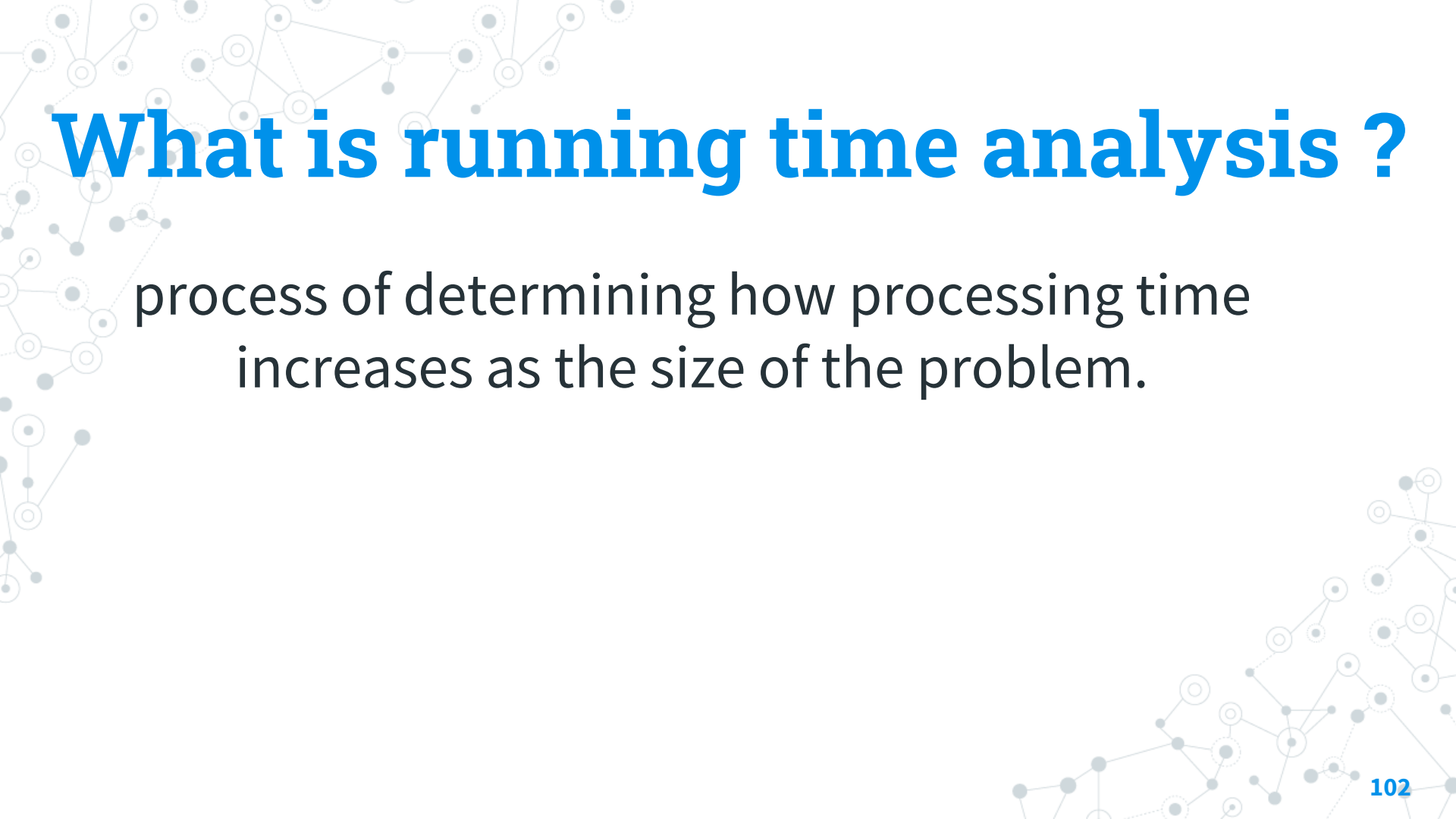
Example Problems

1. Traveling Salesman
2. Image Compression
3. Machine Learning



Algorithm Analysis Criteria

1. correctness
2. speed
3. resources
4. processors
5. developer effort

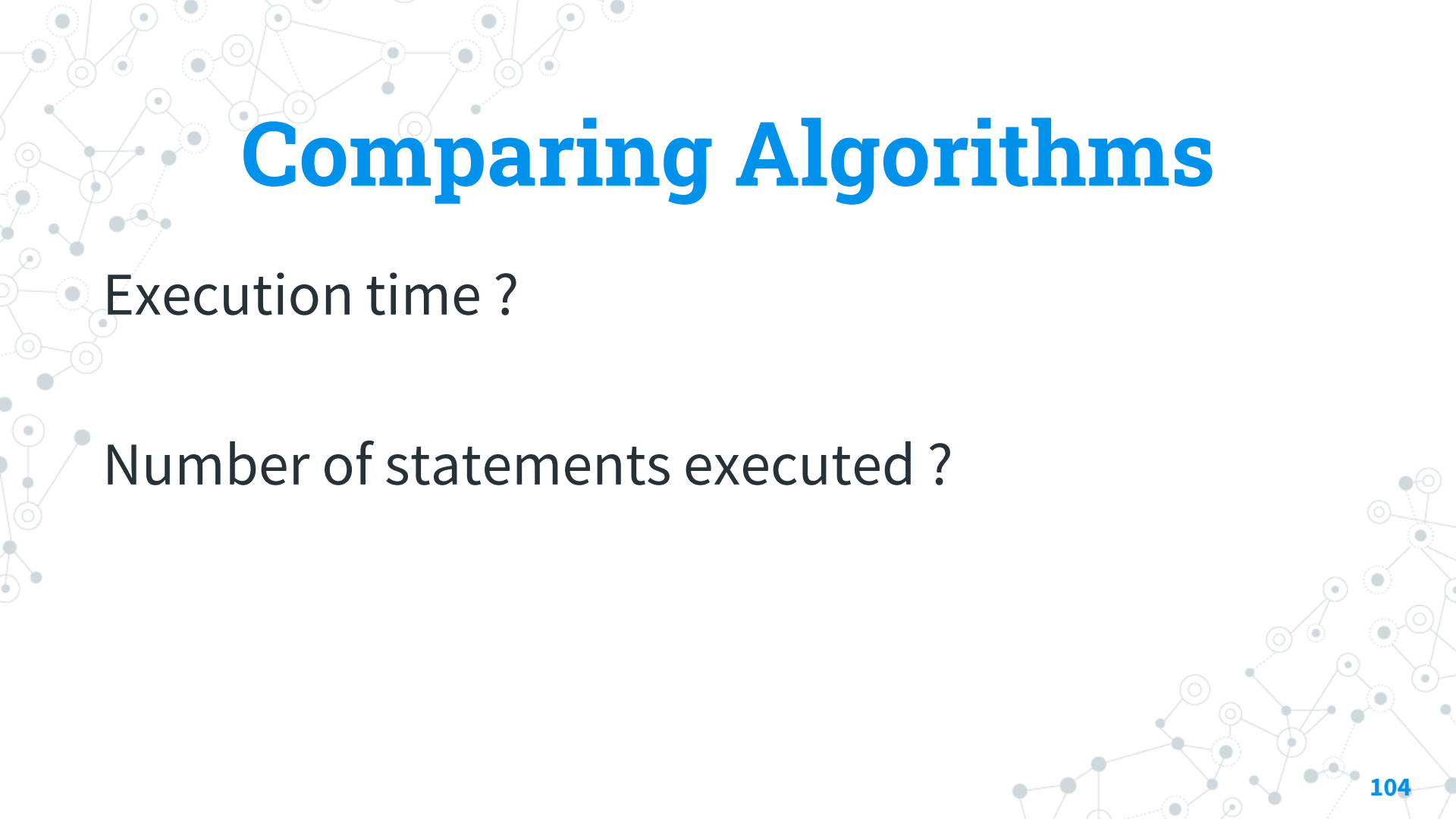


What is running time analysis ?

process of determining how processing time increases as the size of the problem.

Common Types of Input

- * Size of an array
- * Polynomial degree
- * Number of elements in a matrix
- * Number of bits in the binary representation of the input
- * Vertices and edges in a graph.



Comparing Algorithms

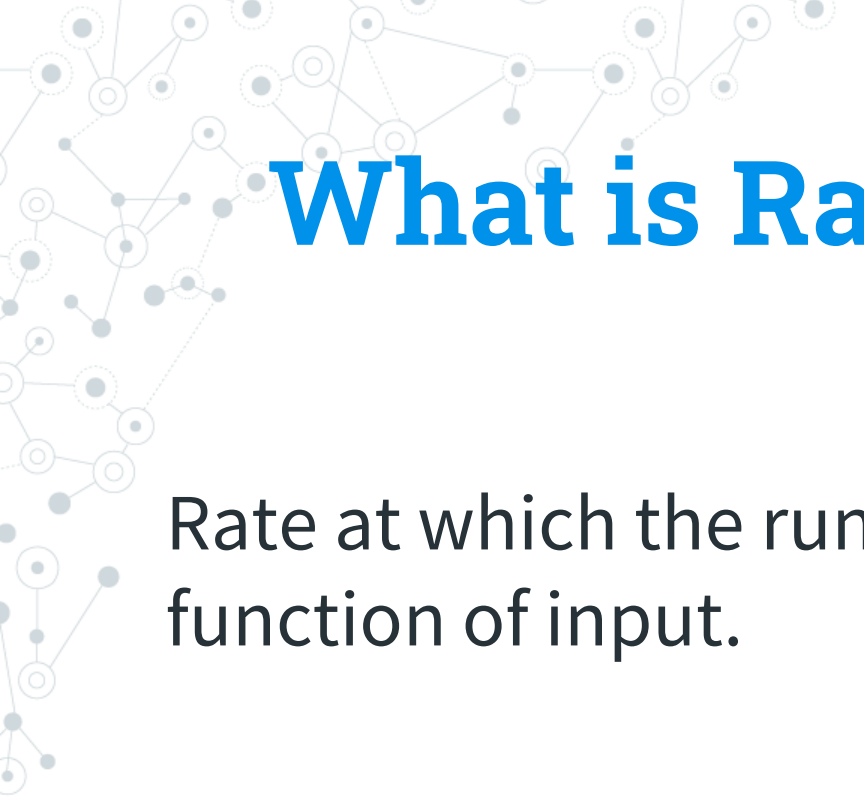
Execution time ?

Number of statements executed ?

Comparing Algorithms

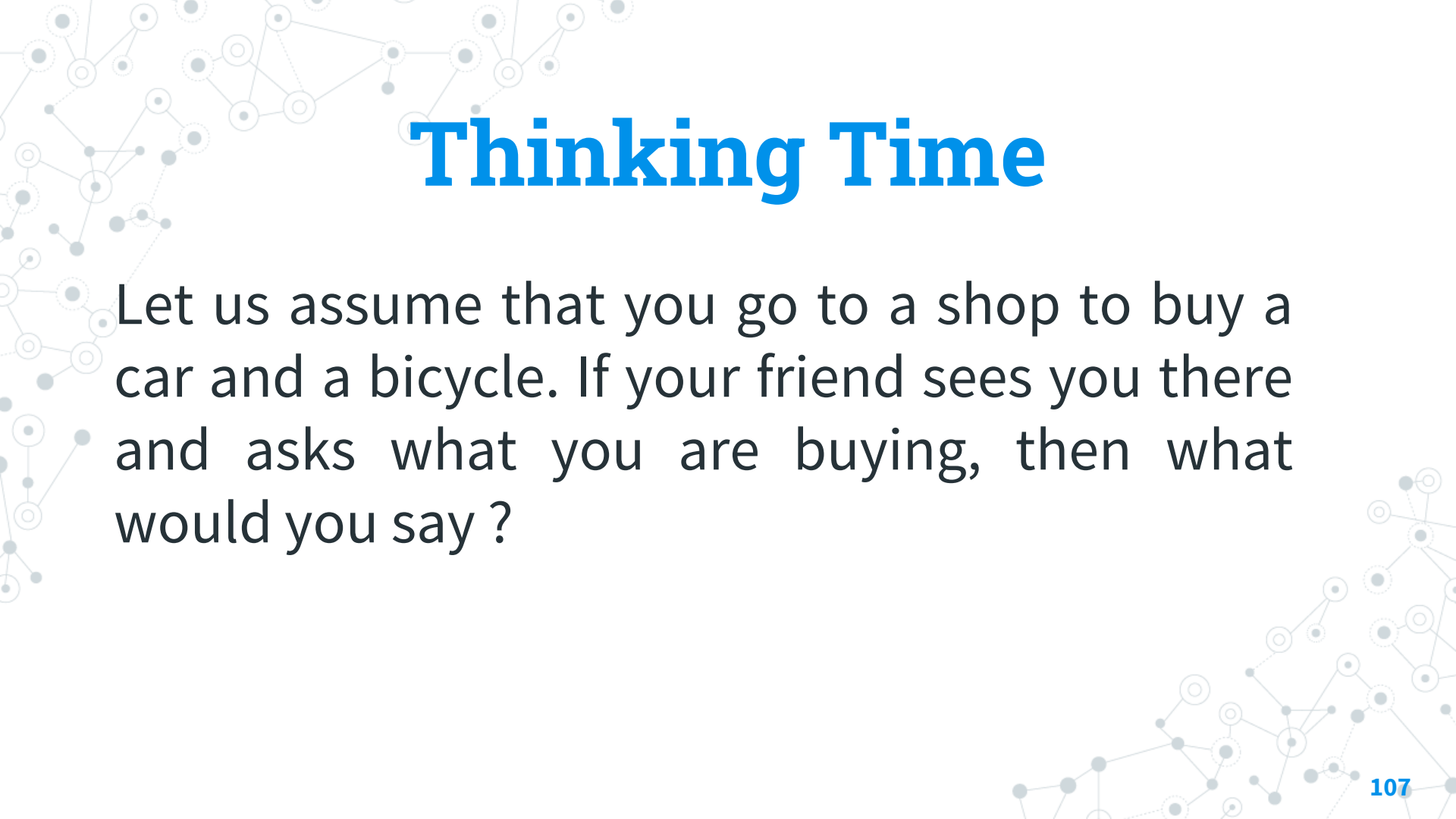
Express the running time of a given algorithm as a function of the input size **$f(n)$** .

Must be independent of machine time, programming style, etc.

A decorative network diagram in the top-left corner, consisting of a series of interconnected nodes and lines, some solid and some dashed, in shades of gray and blue.

What is Rate of Growth ?

Rate at which the running time increases as a function of input.



Thinking Time

Let us assume that you go to a shop to buy a car and a bicycle. If your friend sees you there and asks what you are buying, then what would you say ?

Thinking Time

In general, you say buying a car.

Total Cost = cost_of_car + cost_of_bicycle
Total Cost $\approx$ cost_of_car (approximation)

What is Rate of Growth ?

$$n^4 + 2n^2 + 100n + 500 \approx n^4$$

Rate of Growths

2^{2^n}
$n!$
4^n
2^n
n^2
$n \log n$
$\log(n!)$
n
$2^{\log n}$
$\log^2 n$
$\sqrt{\log n}$
$\log \log n$
1

Rate of Growth ($n = 8$)

2^{2^n}	1.158×10^{77}
$n!$	40320
4^n	65536
2^n	256
n^2	64
$n \log n$	24
$\log(n!)$	15.299208018
n	8
$2^{\log n}$	8
$\log^2 n$	9
$\sqrt{\log n}$	1.73205080757
$\log \log n$	1.584962501
1	1

Rate of Growth ($n = 16$)

2^{2^n}	$2^{(2.092279e+13)}$
$n!$	$2.092279e+13$
4^n	4294967296
2^n	65536
n^2	256
$n \log n$	64
$\log(n!)$	44.523
n	16
$2^{\log n}$	16
$\log^2 n$	16
$\sqrt{\log n}$	2
$\log \log n$	2
1	1



Time Complexity	Name	Example
1	Constant	Adding an element to the front of a linked list
$\log n$	Logarithmic	Finding an element in a sorted array
n	Linear	Finding an element in an unsorted array
$n \log n$	Linear Logarithmic	Sorting n items by 'divide-and-conquer' - Mergesort
n^2	Quadratic	Shortest path between two nodes in a graph
n^3	Cubic	Matrix Multiplication
2^n	Exponential	The Towers of Hanoi problem



Finding an Element in a Sorted Array : Binary Search

Search 23

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

23 > 16
take 2nd half

L=0	1	2	3	M=4	5	6	7	8	H=9
2	5	8	12	16	23	38	56	72	91

23 > 56
take 1st half

0	1	2	3	4	L=5	6	M=7	8	H=9
2	5	8	12	16	23	38	56	72	91

Found 23,
Return 5

0	1	2	3	4	L=5, M=5	H=6	7	8	9
2	5	8	12	16	23	38	56	72	91

Binary Search Analysis

At Iteration 1:

Length of array = n

At Iteration 2:

Length of array = $n/2$

At Iteration 3:

Length of array = $(n/2)/2 = n/2^2$

Therefore, after Iteration k :

Length of array = $n/2^k$

Also, we know that after **k iterations**, the **length of the array becomes 1**

Therefore, the Length of the array

$$n/2^k = 1$$

$$\Rightarrow n = 2^k$$

Applying log function on both sides:

$$\Rightarrow \log_2 n = \log_2 2^k$$

Power Rule

$$\log_a x^p = p \log_a x$$

$$\Rightarrow \log_2 n = k * \log_2 2$$

As **$(\log_a(a) = 1)$** Therefore, **$k = \log_2(n)$**

Types of Analysis

Worst case

- defines the input for which the algorithm takes a long time.
- Input is the one for which the algorithm runs the slowest.

Best case

- defines the input for which the algorithm takes the least time.
- Input is the one for which the algorithm runs the fastest.

Average case

- Provides a prediction about the running time of the algorithm.
- Run the algorithm many times, using many different inputs that come from some distribution that generates these inputs, compute the total running time (by adding the individual times), and divide by the number of trials.
- Assumes that the input is random

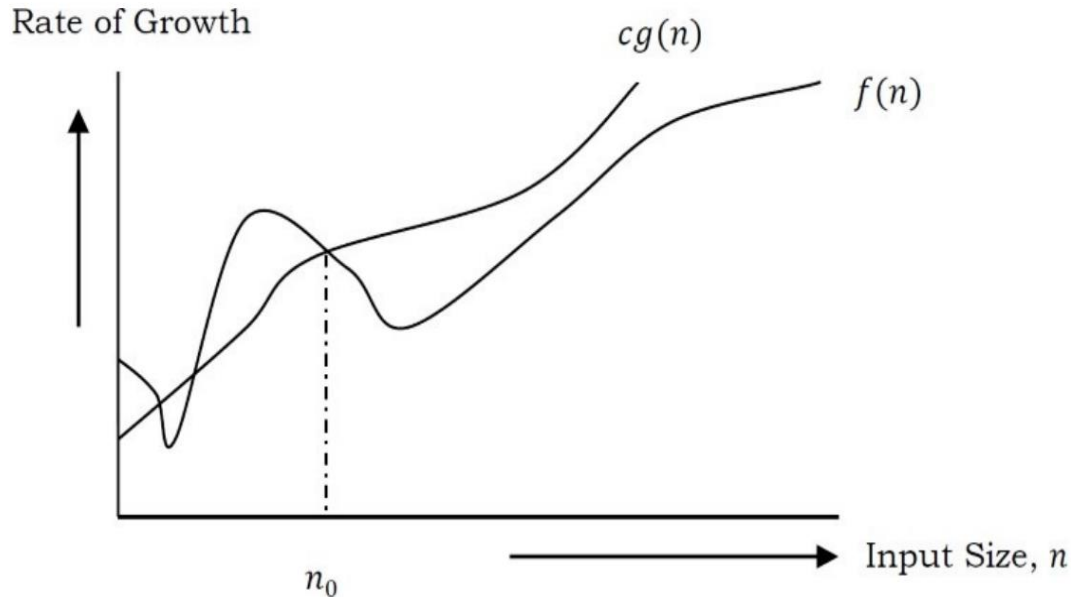
Asymptotic Notation

1. - universal way to measure the speed and efficiency of the algorithm.
2. - describes the behavior of the function as its input approaches infinity.
3. **Big-O Notation** (Upper Bounding Function)
4. **Omega-Q Notation** (Lower Bounding Function)
5. **Theta Notation** (Order Function)

Big-O Notation (Upper Bounding Function)

$$f(n) = O(g(n))$$

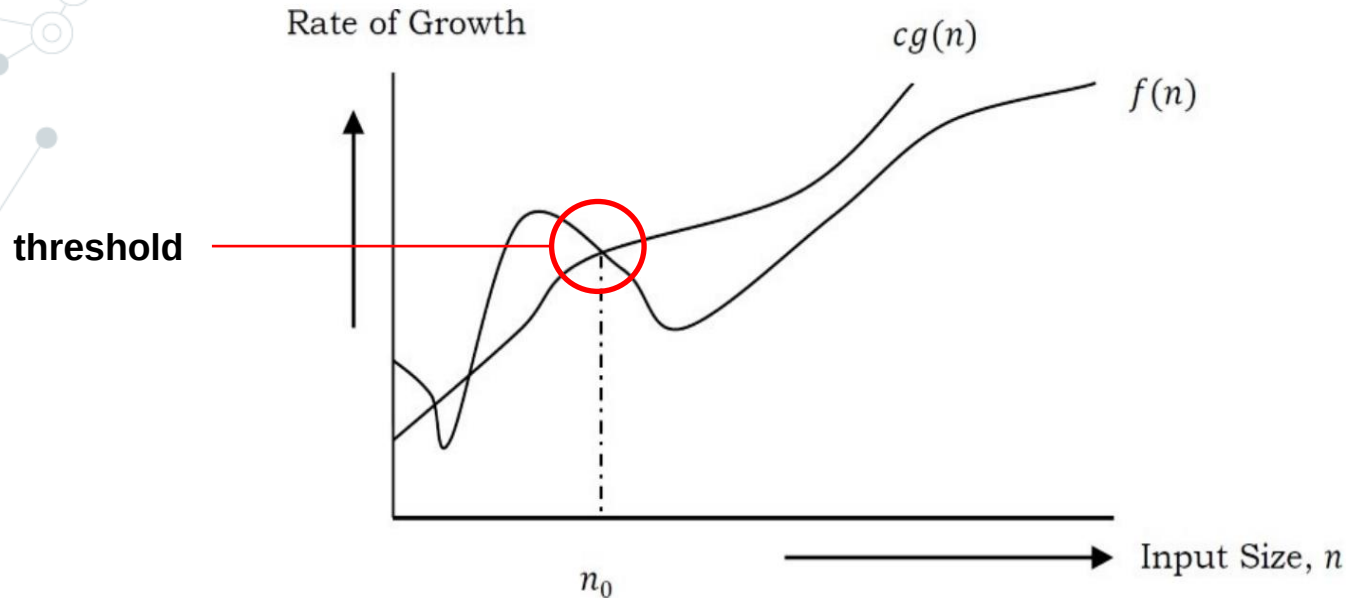
at larger values of n , the upper bound of $f(n)$ is $g(n)$.



Big-O Notation (Upper Bounding Function)

$$f(n) = O(g(n))$$

at larger values of n , the upper bound of $f(n)$ is $g(n)$.



Big-O Notation (Upper Bounding Function)

1. $O(g(n)) = \{$

1. $f(n) : \exists$ positive constants c and n_0

2. $: 0 \leq f(n) \leq cg(n) \forall n > n_0$

2. $\}$

Big-O Notation (Upper Bounding Function)

⊙ $f(n) = n^4 + 100n^2 + 10n + 50$

⊙ $g(n) = ?$

Big-O Notation (Upper Bounding Function)

⊙ $f(n) = n^4 + 100n^2 + 10n + 50$

⊙ $g(n) = n^4$

Big-O Notation (Upper Bounding Function)

$O(1)$: 100, 1000, 200, 1, 20, *etc.*

$O(n)$: $3n + 100$, $100n$, $2n - 1$, 3, *etc.*

$O(n \log n)$: $5n \log n$, $3n - 100$, $2n - 1$, 100, $100n$, *etc.*

$O(n^2)$: n^2 , $5n - 10$, 100, $n^2 - 2n + 1$, 5, *etc.*

Big-O Examples

Example-1 :

Find an upper bound for $f(n) = 3n + 8$

Big-O Examples

Example-1 : Find an upper bound for $f(n) = 3n + 8$

Answer : $3n + 8 \leq 4n$, for all $n \geq 8$, with $c = 4$
and $n_0 = 8$

Big-O Examples

Example-1 :

Find upper bound for $f(n) = 3n + 8$

Answer :

$3n + 8 \leq 4n$, for all $n \geq 8$

with $c = 4$ and $n_0 = 8$

n	$3n + 8$	$4n$	$3n + 8 < 4n$	Result
1	11	4	$11 < 4$	False
2	14	8	$14 < 8$	False
3	17	12	$17 < 12$	False
4	20	16	$20 < 16$	False
5	23	20	$23 < 20$	False
6	26	24	$26 < 24$	False
7	29	28	$29 < 28$	False
8	32	32	$32 < 32$	True
9	35	36	$35 < 36$	True
10	38	40	$38 < 40$	True
11	41	44	$41 < 44$	True
12	44	48	$44 < 48$	True
13	47	52	$47 < 52$	True
14	50	56	$50 < 56$	True
15	53	60	$53 < 60$	True

Big-O Examples

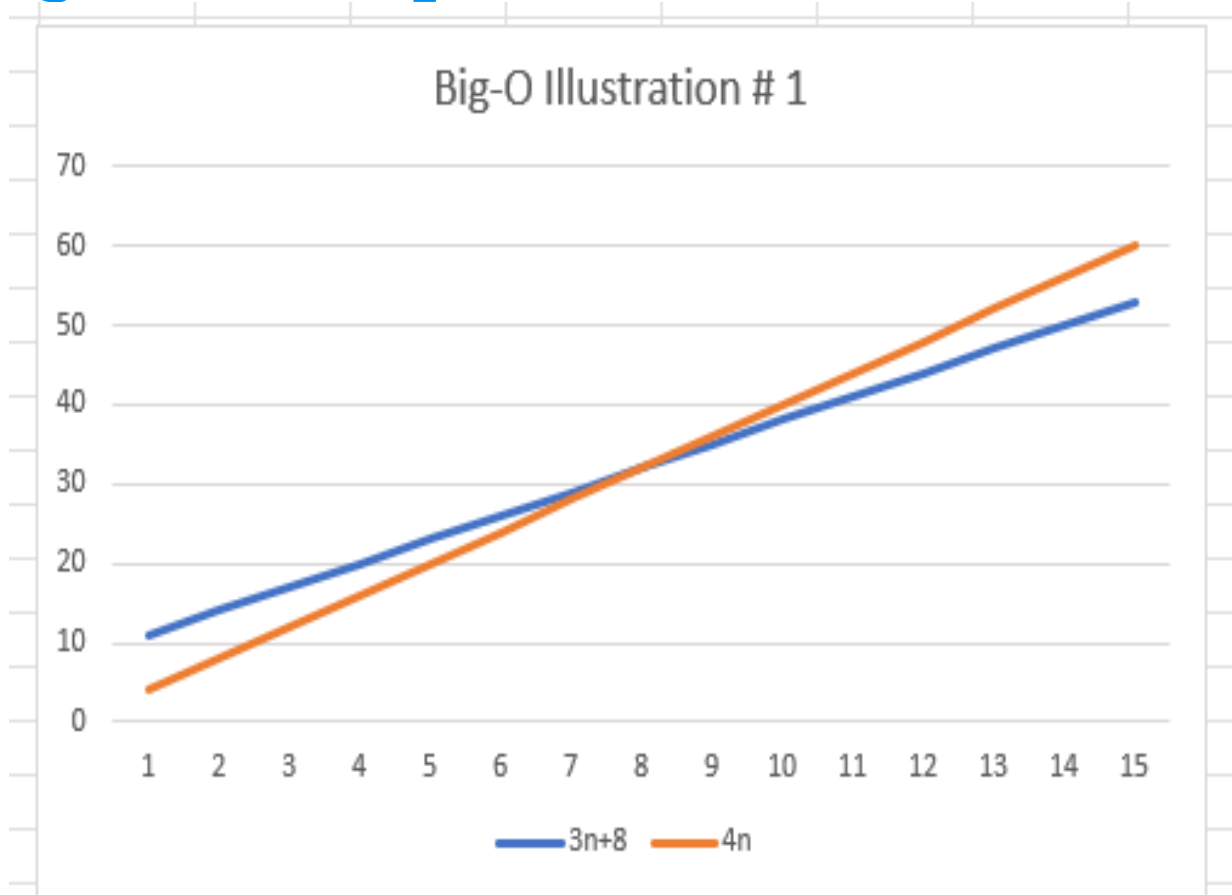
Example-1 :

Find upper bound for $f(n) = 3n + 8$

Answer :

$3n + 8 \leq 4n$, for all $n \geq 8$

with $c = 4$ and $n_0 = 8$



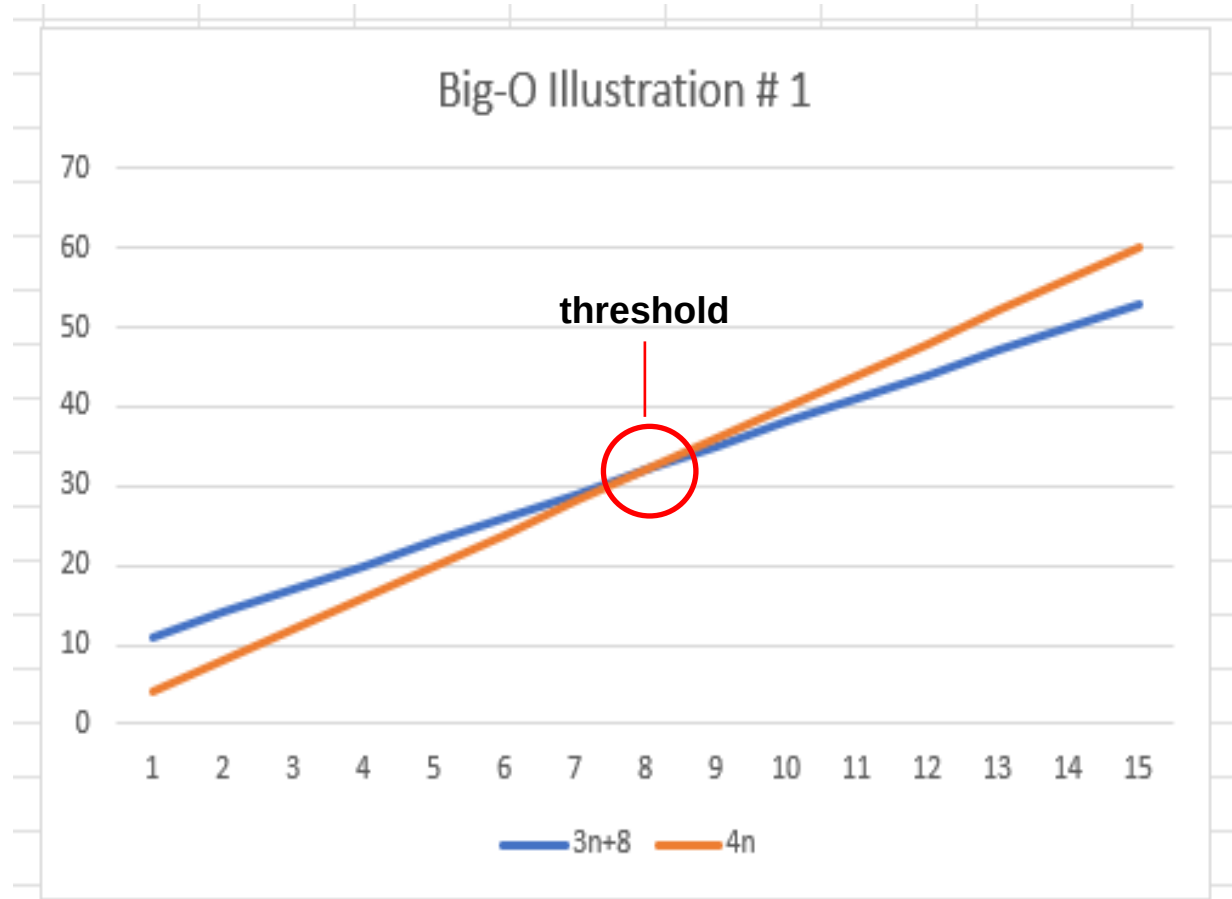
Big-O Examples

Example-1 :

Find upper bound for $f(n) = 3n + 8$

Answer :

$3n + 8 \leq 4n$, for all $n \geq 8$
with $c = 4$ and $n_0 = 8$



Big-O Examples

Example-2 : Find upper bound for $f(n) = n^2 + 1$

Answer : $n^2 + 1 \leq 2n^2$, for all $n \geq 1$

$\therefore n^2 + 1 = O(n^2)$ with $c = 2$ and $n_0 = 1$

Big-O Examples

Example-2 : Find upper bound for $f(n) = n^2 + 1$

Answer :

$$n^2 + 1 \leq 2n^2, \text{ for all } n \geq 1$$

$$\therefore n^2 + 1 = O(n^2) \text{ with } c = 2 \text{ and } n_0 = 1$$

n	$n^2 + 1$	$2n^2$	$n^2 + 1 \leq 2n^2$	Result
1	2	2	$2 \leq 2$	True
2	5	8	$5 \leq 8$	True
3	10	18	$10 \leq 18$	True
4	17	32	$17 \leq 32$	True
5	26	50	$26 \leq 50$	True
6	37	72	$37 \leq 72$	True
7	50	98	$50 \leq 98$	True
8	65	128	$65 \leq 128$	True
9	82	162	$82 \leq 162$	True
10	101	200	$101 \leq 200$	True
11	122	242	$122 \leq 242$	True
12	145	288	$145 \leq 288$	True
13	170	338	$170 \leq 338$	True
14	197	392	$197 \leq 392$	True
15	226	450	$226 \leq 450$	True

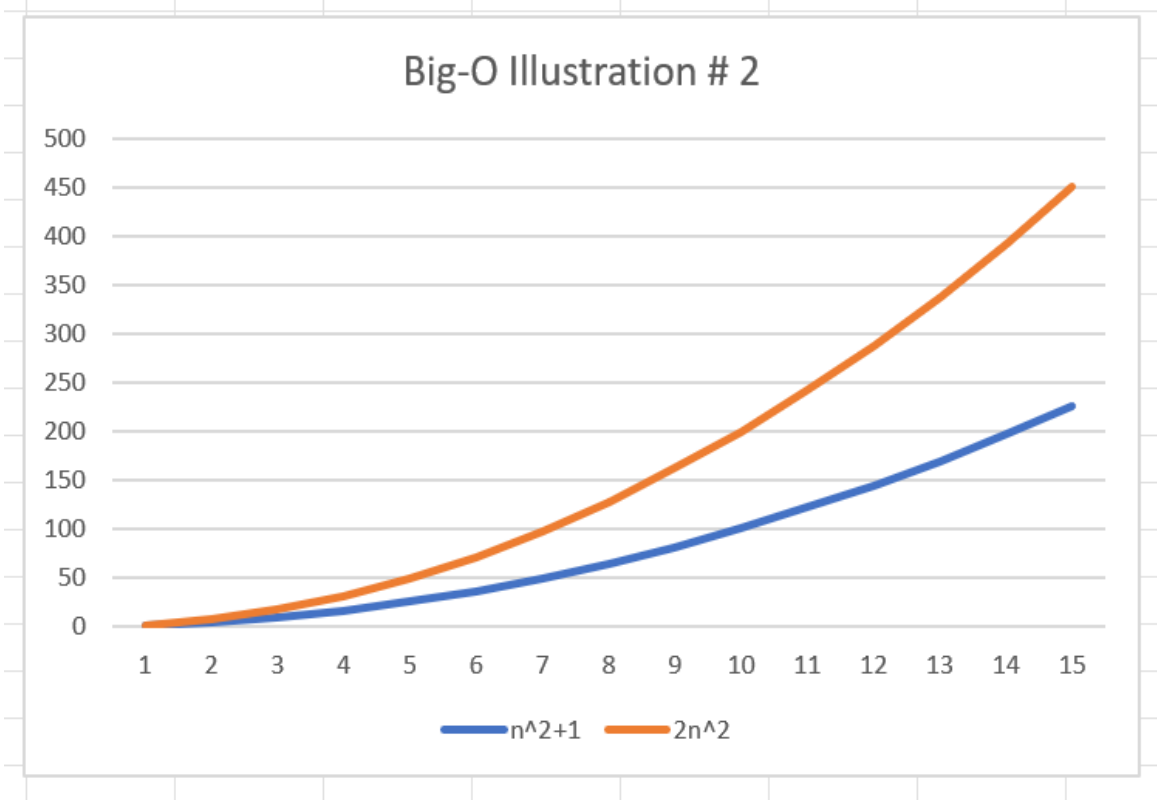
Big-O Examples

Example-2 : Find upper bound for $f(n) = n^2 + 1$

Solution :

$$n^2 + 1 \leq 2n^2, \text{ for all } n \geq 1$$

$$\therefore n^2 + 1 = O(n^2) \text{ with } c = 2 \text{ and } n_0 = 1$$



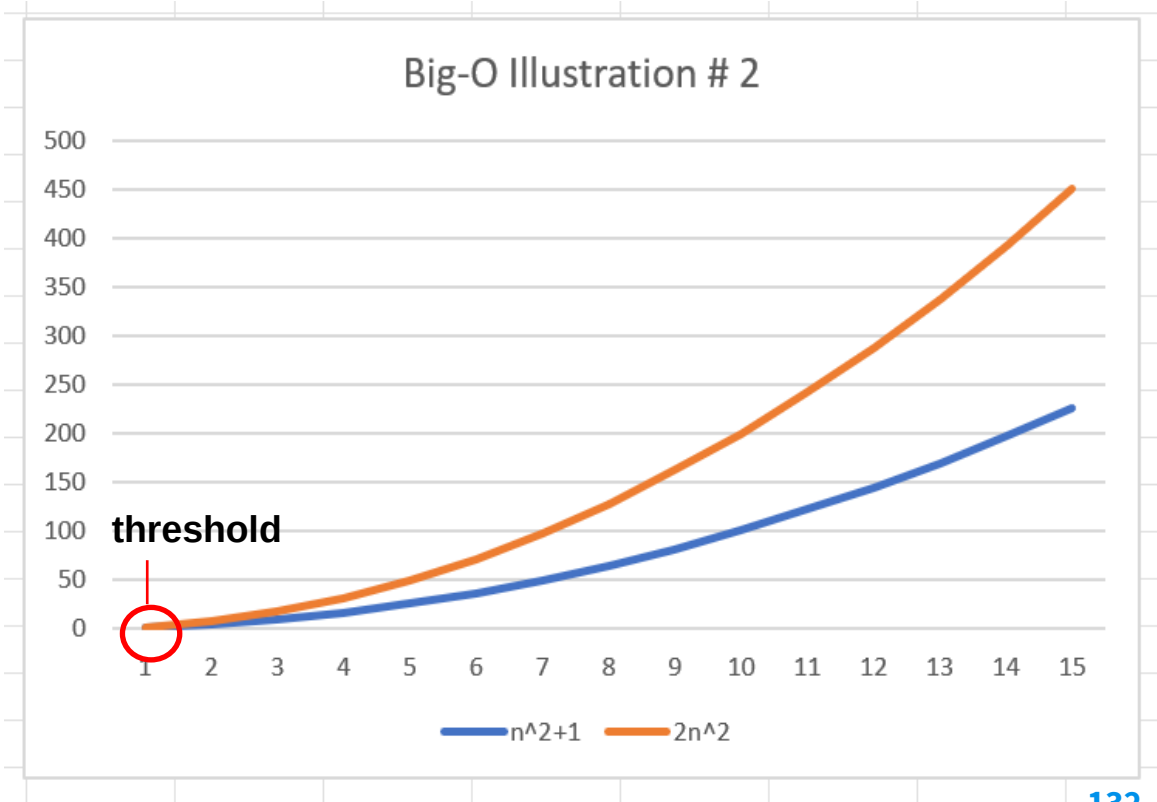
Big-O Examples

Example-2 : Find upper bound for $f(n) = n^2 + 1$

Solution :

$$n^2 + 1 \leq 2n^2, \text{ for all } n \geq 1$$

$$\therefore n^2 + 1 = O(n^2) \text{ with } c = 2 \text{ and } n_0 = 1$$

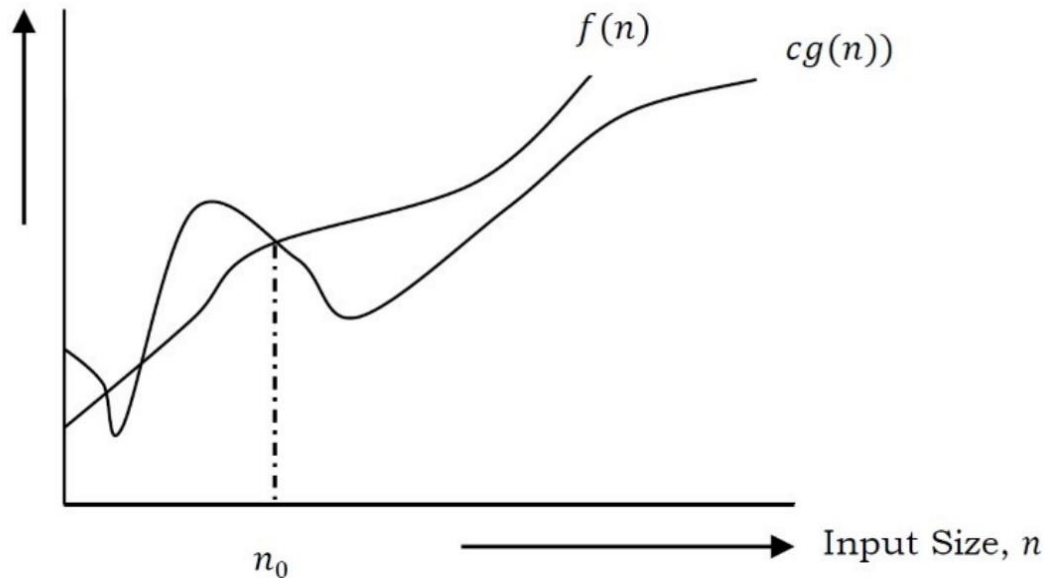


Omega-Q Notation (Lower Bounding Function)

$$f(n) = \Omega(g(n))$$

at larger values of n , the tighter lower bound of $f(n)$ is $g(n)$.

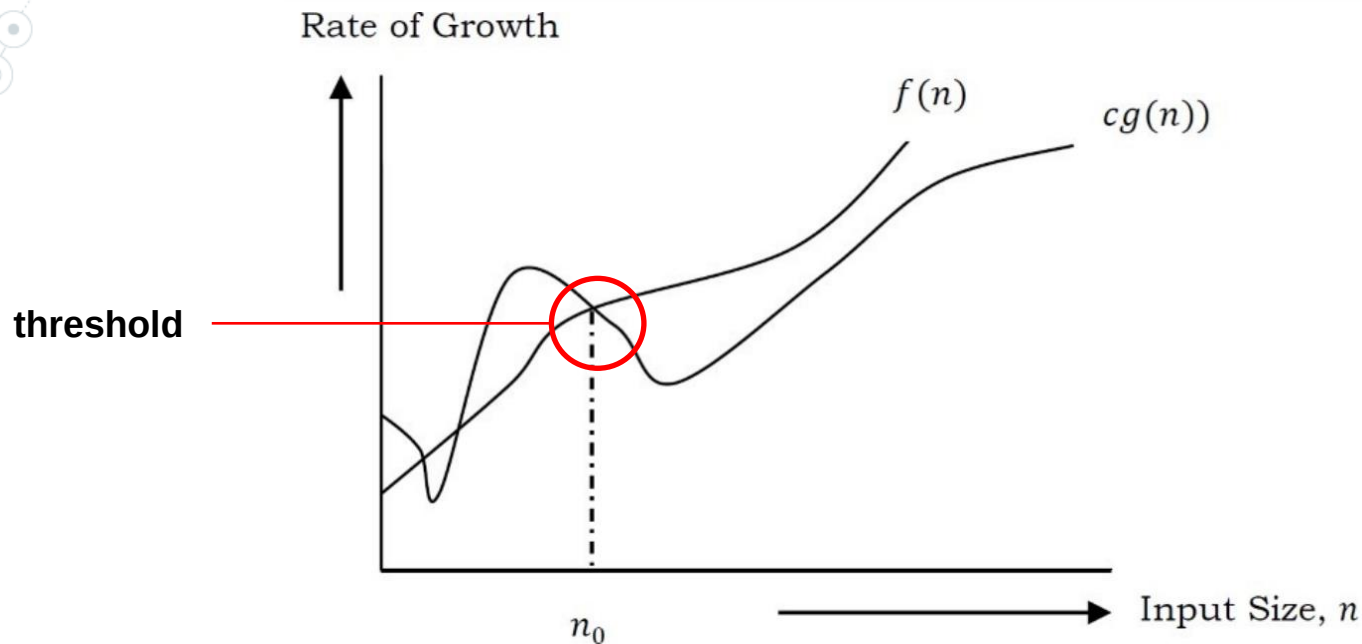
Rate of Growth



Omega-Q Notation (Lower Bounding Function)

$$f(n) = \Omega(g(n))$$

at larger values of n , the tighter lower bound of $f(n)$ is $g(n)$.



Omega-Q Notation (Lower Bounding Function)

$$\Omega(g(n)) = \{$$

1. $f(n) : \exists$ positive constants c and n_0
2. $: 0 \leq cg(n) \leq f(n) \forall n \geq n_0$

$$2. \}$$

Omega-Q Examples

Example-1 : Find lower bound for $f(n) = 5n^2$

Answer: $\exists c, n_0 : 0 \leq cn^2 \leq 5n^2$

$$\Rightarrow cn^2 \leq 5n^2 \Rightarrow c = 5 \text{ and } n_0 = 1$$

$$\therefore 5n^2 = \Omega(n^2) \text{ with } c = 5 \text{ and } n_0 = 1$$

Omega-Q Examples

Example-1 : Find lower bound for $f(n) = 5n^2$

Answer :

$$\exists c, n_0 : 0 \leq cn^2 \leq 5n^2$$

$$\Rightarrow cn^2 \leq 5n^2 \Rightarrow c = 5 \text{ and } n_0 = 1$$

$$\therefore 5n^2 = \Omega(n^2) \text{ with } c = 5 \text{ and } n_0 = 1$$

n	cn^2	$5n^2$	$cn^2 \leq 5n^2$	Result
1	6	6	$6 \leq 6$	True
2	21	21	$21 \leq 21$	True
3	46	46	$46 \leq 46$	True
4	81	81	$81 \leq 81$	True
5	126	126	$126 \leq 126$	True
6	181	181	$181 \leq 181$	True
7	246	246	$246 \leq 246$	True
8	321	321	$321 \leq 321$	True
9	406	406	$406 \leq 406$	True
10	501	501	$501 \leq 501$	True
11	606	606	$606 \leq 606$	True
12	721	721	$721 \leq 721$	True
13	846	846	$846 \leq 846$	True
14	981	981	$981 \leq 981$	True
15	1126	1126	$1126 \leq 1126$	True

Omega-Q Examples

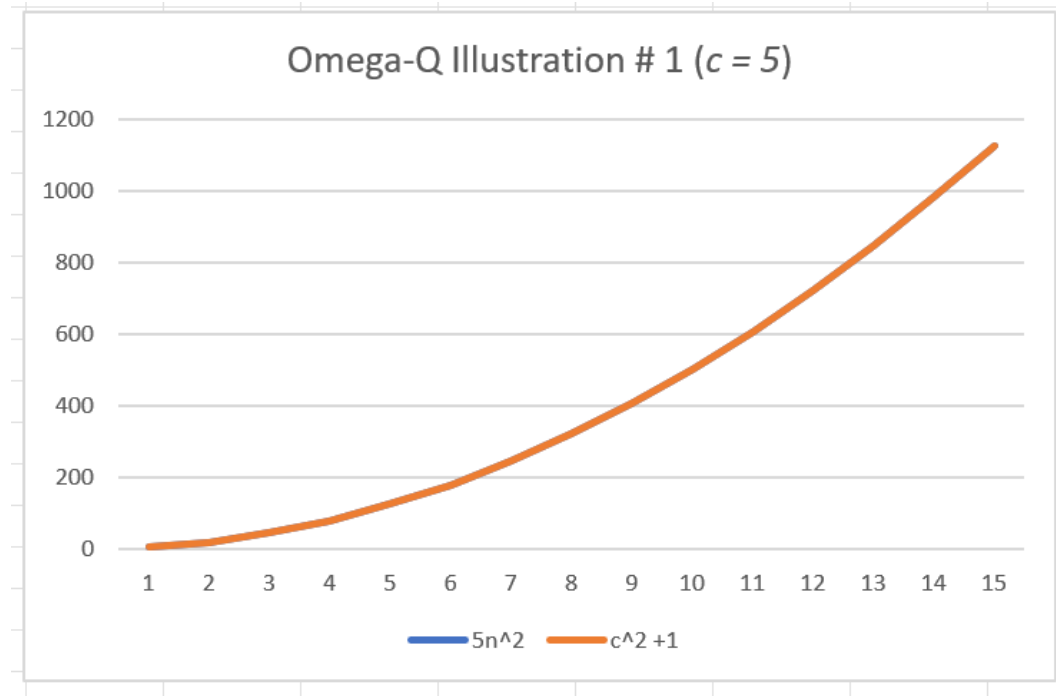
Example-1 : Find lower bound for $f(n) = 5n^2$

Solution :

$$\exists c, n_0 : 0 \leq cn^2 \leq 5n^2$$

$$\Rightarrow cn^2 \leq 5n^2 \Rightarrow c = 5 \text{ and } n_0 = 1$$

$$\therefore 5n^2 + 8 = \Omega(n^2) \text{ with } c = 5 \text{ and } n_0 = 1$$



Omega-Q Examples

Example-1 : Find lower bound for $f(n) = 5n^2$

Answer : $\exists c, n_0 : 0 \leq cn^2 \leq 5n^2$

$$\Rightarrow cn^2 \leq 5n^2 \Rightarrow c = 4 \text{ and } n_0 = 1$$

$$\therefore 5n^2 + 8 = \Omega(n^2) \text{ with } c = 4 \text{ and } n_0 = 1$$

Omega-Q Examples

Example-1 : Find lower bound for $f(n) = 5n^2$

Answer :

$$\exists c, n_0 : 0 \leq cn^2 \leq 5n^2$$

$$\Rightarrow cn^2 \leq 5n^2 \Rightarrow c = 4 \text{ and } n_0 = 1$$

$$\therefore 5n^2 + 8 = \Omega(n^2) \text{ with } c = 4 \text{ and } n_0 = 1$$

n	cn^2	$5n^2$	$cn^2 \leq 5n^2$	Result
1	5	5	$5 \leq 5$	True
2	16	20	$16 \leq 20$	True
3	36	45	$36 \leq 45$	True
4	64	80	$64 \leq 80$	True
5	100	125	$100 \leq 125$	True
6	144	180	$144 \leq 180$	True
7	196	245	$196 \leq 245$	True
8	256	320	$256 \leq 320$	True
9	324	405	$324 \leq 405$	True
10	400	500	$400 \leq 500$	True
11	484	605	$484 \leq 605$	True
12	576	720	$576 \leq 720$	True
13	676	845	$676 \leq 845$	True
14	784	980	$784 \leq 980$	True
15	900	1125	$900 \leq 1125$	True

Omega-Q Examples

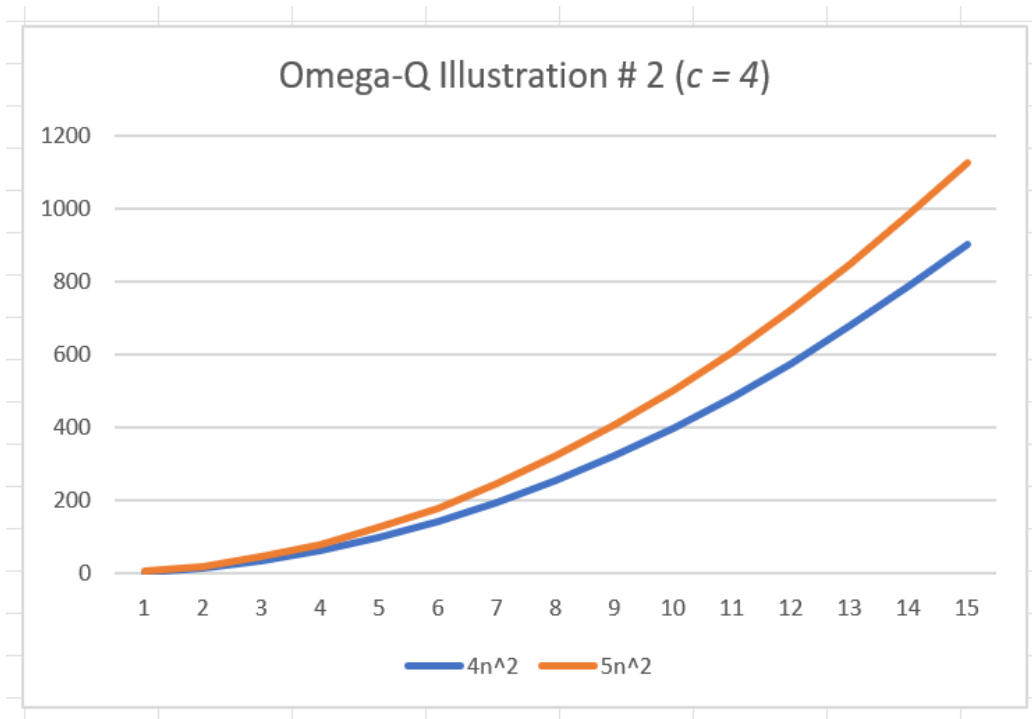
Example-1 : Find lower bound for $f(n) = 5n^2$

Solution :

$$\exists c, n_0 : 0 \leq cn^2 \leq 5n^2$$

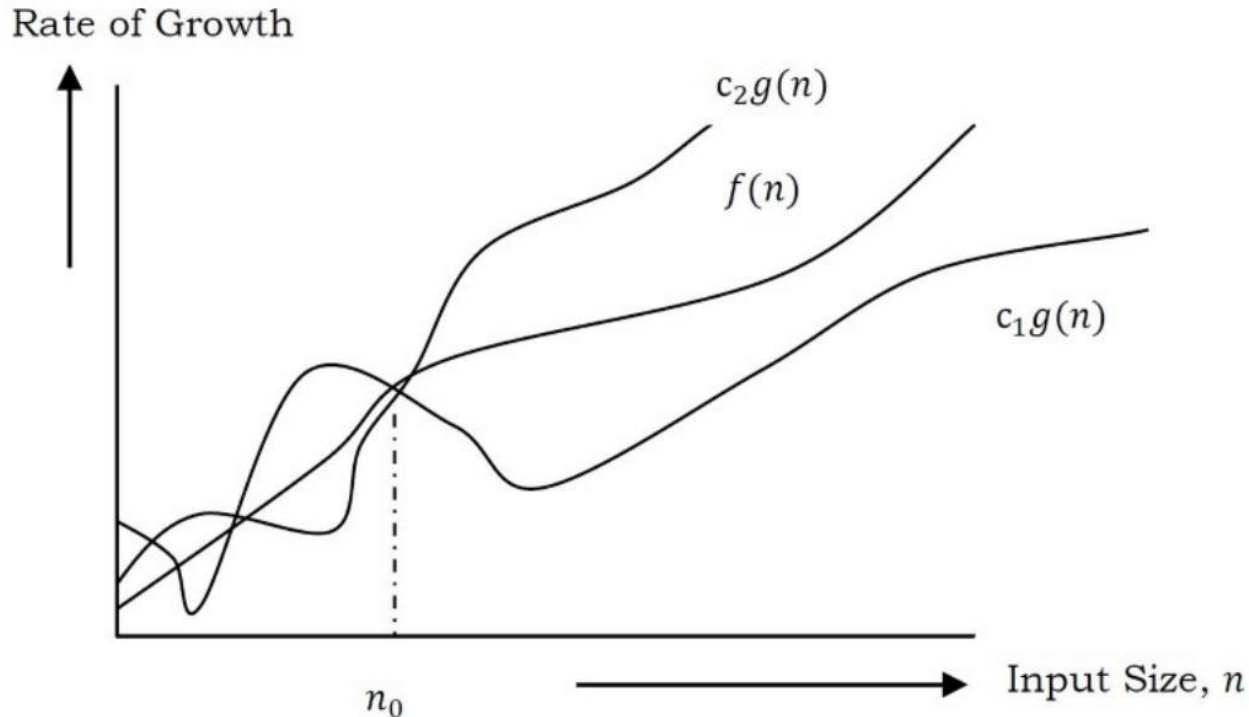
$$\Rightarrow cn^2 \leq 5n^2 \Rightarrow c = 4 \text{ and } n_0 = 1$$

$$\therefore 5n^2 + 8 = \Omega(n^2) \text{ with } c = 4 \text{ and } n_0 = 1$$



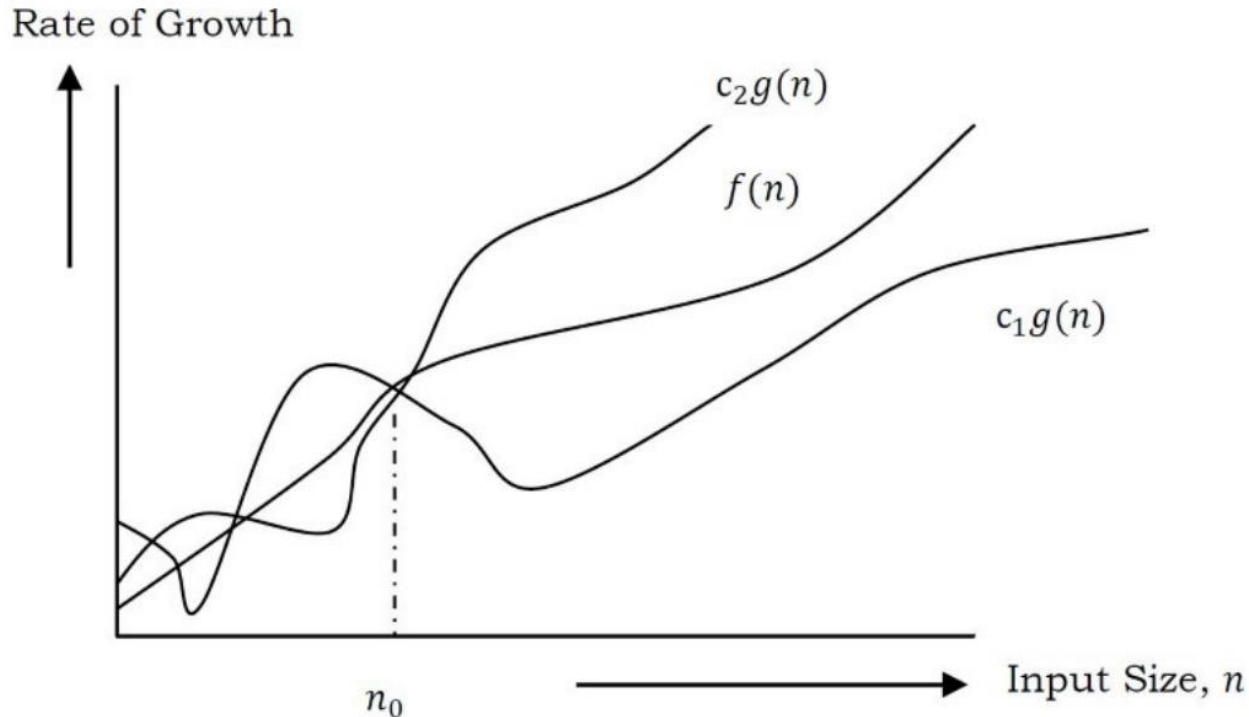
Theta- Θ Notation (Order Function)

The average running time of an algorithm is always between the lower bound and the upper bound.



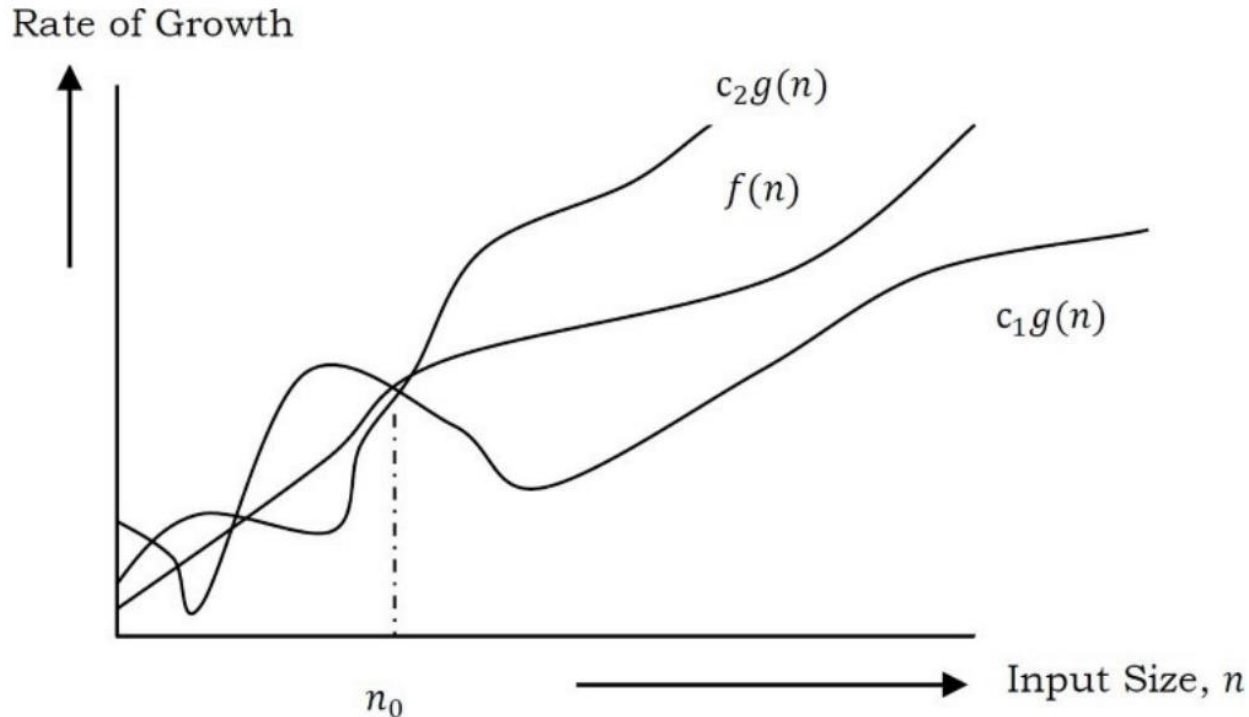
Theta- Θ Notation (Order Function)

If the **upper bound** (O) and **lower bound** (Ω) give the **same** result, then the **Θ notation** will also have the **same** rate of growth.



Theta- Θ Notation (Order Function)

If the **upper bound** (O) and **lower bound** (Ω) give a **different** result, then the **Θ notation may not have the same** rate of growth.



Theta- Θ Notation (Order Function)

1. $\Theta(g(n)) = \{$

1. $f(n) : \exists$ positive constants c_1, c_2 and n_0

2. $: 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$

2. $\}$

Theta-Θ Examples

Example-1 : Find Θ bound for $f(n) = (n^2/2) - (n/2)$

Answer : $(n^2/5) \leq (n^2/2) - (n/2) \leq n^2 \quad \forall n \geq 2$

$\therefore (n^2/2) - (n/2) = \Theta(n^2)$ with $c_1 = 1/5, c_2 = 1, n_0 = 2$

Theta-Θ Examples

Example-1 : Find Θ bound for

$$f(n) = (n^2/2) - (n/2)$$

Answer :

$$(n^2/5) \leq (n^2/2) - (n/2) \leq n^2 \quad \forall n \geq 2$$

$$\therefore (n^2/2) - (n/2) = \Theta(n^2) \text{ with } c_1 = 1/5, c_2 = 1, n_0 = 2$$

n	$(n^2)/5$	$((n^2)/2) - (n/2)$	n^2	$(n^2)/5 \leq ((n^2)/2) - (n/2) \leq n^2$	Result
1	0.2	0	1	$0.2 \leq 0 \leq 1$	False
2	0.8	1	4	$0.8 \leq 1 \leq 4$	True
3	1.8	3	9	$1.8 \leq 3 \leq 9$	True
4	3.2	6	16	$3.2 \leq 6 \leq 16$	True
5	5	10	25	$5 \leq 10 \leq 25$	True
6	7.2	15	36	$7.2 \leq 15 \leq 36$	True
7	9.8	21	49	$9.8 \leq 21 \leq 49$	True
8	12.8	28	64	$12.8 \leq 28 \leq 64$	True
9	16.2	36	81	$16.2 \leq 36 \leq 81$	True
10	20	45	100	$20 \leq 45 \leq 100$	True
11	24.2	55	121	$24.2 \leq 55 \leq 121$	True
12	28.8	66	144	$28.8 \leq 66 \leq 144$	True
13	33.8	78	169	$33.8 \leq 78 \leq 169$	True
14	39.2	91	196	$39.2 \leq 91 \leq 196$	True
15	45	105	225	$45 \leq 105 \leq 225$	True

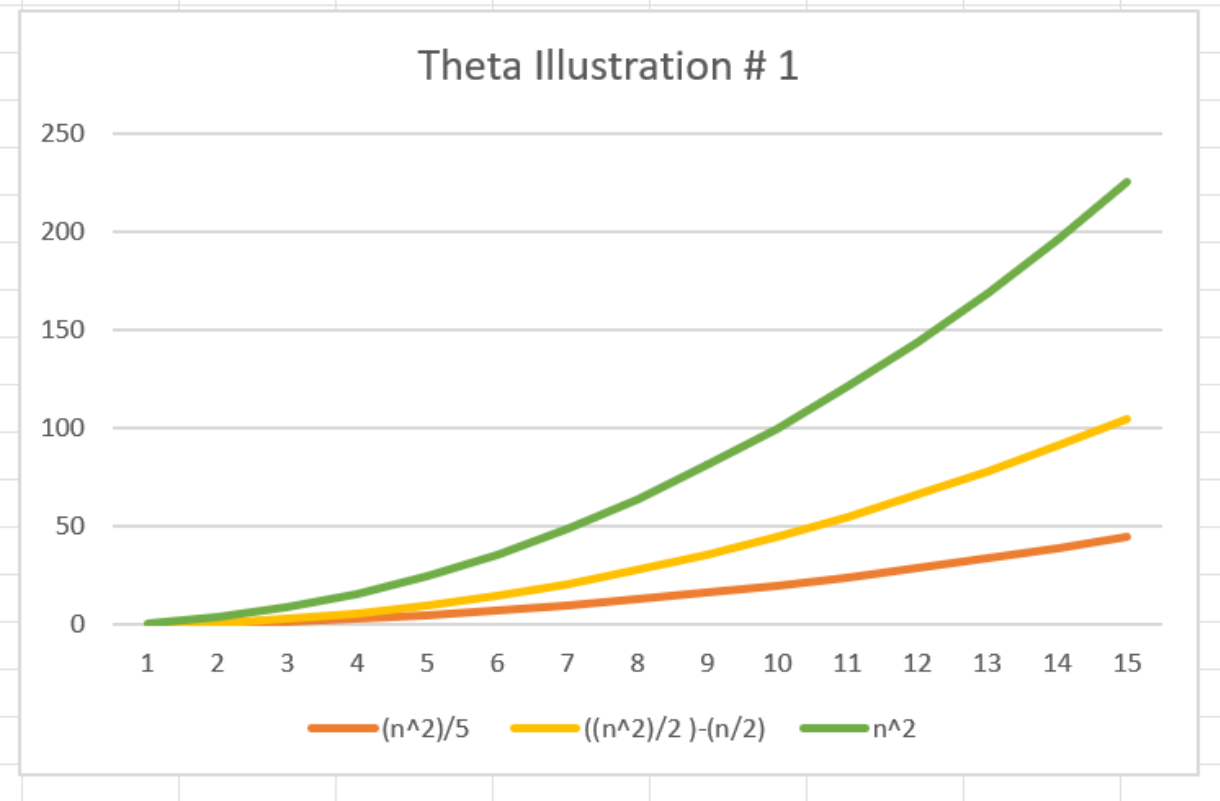
Theta-Θ Examples

Example-1 : Find Θ bound for
 $f(n) = (n^2/2) - (n/2)$

Solution :

$$(n^2/5) \leq (n^2/2) - (n/2) \leq n^2 \quad \forall n \geq 2$$

$$\therefore (n^2/2) - (n/2) = \Theta(n^2) \text{ with } c_1 = 1/5, c_2 = 1, n_0 = 2$$



Determine the running time of an Algorithm

1. Loops :

1. *running time of inside statements * # of iterations*

Determine the running time of an Algorithm

1. Loops :

*running time of inside statements * # of iterations*

1. **1.** *for($i=1; i \leq n; i++$)*

2. *$m = m + 2;$*

Determine the running time of an Algorithm

1. Loops :

*running time of inside statements * # of iterations*

1. **1.** `for(i=1; i <= n; i++)`

2. `m = m + 2;     // constant c time`

Determine the running time of an Algorithm

1. Loops :

*running time of inside statements * # of iterations*

1. **1.** `for(i=1; i <= n; i++)` // **n times**
2. `m = m + 2;` // **constant c time**

Determine the running time of an Algorithm

1. Loops :

*running time of inside statements * # of iterations*

$$\text{Total time} = c * n = cn = O(n)$$

1. 1. *for*($i=1; i \leq n; i++$) // **n** times

2. $m = m + 2;$ // **constant c** time

Determine the running time of an Algorithm

2. Nested Loops :

Product of the sizes of all the loops.

```
1. 1. for(i=1; i <= n; i++){  
2.   for(j=1; j <= n; j++){  
3.     k = k + 1;  
4.   }  
5. }
```

Determine the running time of an Algorithm

2. Nested Loops :

Product of the sizes of all the loops.

1. **1.** `for(i=1; i <= n; i++){`
2. `for(j=1; j <= n; j++){`
3. `k = k + 1; // constant c time`
4. `}`
5. `}`

Determine the running time of an Algorithm

2. Nested Loops :

Product of the sizes of all the loops.

1. **1.** `for(i=1; i <= n; i++){`
2. `for(j=1; j <= n; j++){ //executes n times`
3. `k = k + 1; //constant c time`
4. `}`
5. `}`

Determine the running time of an Algorithm

2. Nested Loops :

Product of the sizes of all the loops.

1. **1.** `for(i=1; i <= n; i++){ //executes n times`
2. `for(j=1; j <= n; j++){ //executes n times`
3. `k = k + 1; //constant c time`
4. `}`
5. `}`

Determine the running time of an Algorithm

2. Nested Loops :

Product of the sizes of all the loops.

Total time = $c * n * n = cn^2 = O(n^2)$

1. **1.** `for(i=1; i <= n; i++){//executes n times`
- 2.** `for(j=1; j <= n; j++){ //executes n times`
- 3.** `k = k + 1; //constant c time`
- 4.** `}`
- 5.** `}`

Determine the running time of an Algorithm

3. Consecutive Statements :

Total time complexities of each statement

Determine the running time of an Algorithm

3. Consecutive Statements :

1. **1.** $x = x + 1;$
2. **2.** $\text{for}(i = 1; i \leq n; i++)$
3. **3.** $m = m + 1;$
4. **4.** $\text{for}(i = 1; i \leq n; i++)\{$
5. **5.** $\text{for}(j = 1; j \leq n; j++)$
6. **6.** $k = k + 1;$
7. **7.** $\}$

Determine the running time of an Algorithm

3. Consecutive Statements :

1. **1.** $x = x + 1;$ // **constant c_0 time**
- 2.** $\text{for}(i=1; i \leq n; i++)$
- 3.** $m = m + 1;$
- 4.** $\text{for}(i=1; i \leq n; i++)\{$
- 5.** $\text{for}(j=1; j \leq n; j++)$
- 6.** $k = k + 1;$
- 7.** $\}$

Determine the running time of an Algorithm

3. Consecutive Statements :

1. **1.** $x = x + 1;$ // **constant c_0 time**
2. **2.** $\text{for}(i=1; i \leq n; i++)$ // **executed n times**
3. **3.** $m = m + 1;$
4. **4.** $\text{for}(i=1; i \leq n; i++)\{$
5. **5.** $\text{for}(j=1; j \leq n; j++)$
6. **6.** $k = k + 1;$
7. **7.** $\}$

Determine the running time of an Algorithm

3. Consecutive Statements :

1. **1.** $x = x + 1;$ // **constant c_0 time**
- 2.** $\text{for}(i = 1; i \leq n; i++)$ // **executed n times**
- 3.** $m = m + 1;$ // **constant c_1 time**
- 4.** $\text{for}(i = 1; i \leq n; i++)\{$
- 5.** $\text{for}(j = 1; j \leq n; j++)$
- 6.** $k = k + 1;$
- 7.** $\}$

Determine the running time of an Algorithm

3. Consecutive Statements :

1. **1.** $x = x + 1;$ // **constant c_0 time**
2. **2.** $\text{for}(i = 1; i \leq n; i++)$ // **executed n times**
3. **3.** $m = m + 1;$ // **constant c_1 time**
4. **4.** $\text{for}(i = 1; i \leq n; i++)\{$ // **executed n times**
5. **5.** $\text{for}(j = 1; j \leq n; j++)$
6. **6.** $k = k + 1;$
7. **7.** $\}$

Determine the running time of an Algorithm

3. Consecutive Statements :

1. **1.** $x = x + 1;$ // **constant c_0 time**
2. **2.** $\text{for}(i = 1; i \leq n; i++)$ // **executed n times**
3. **3.** $m = m + 1;$ // **constant c_1 time**
4. **4.** $\text{for}(i = 1; i \leq n; i++)\{$ // **executed n times**
5. **5.** $\text{for}(j = 1; j \leq n; j++)$ // **executed n times**
6. **6.** $k = k + 1;$
7. **7.** $\}$

Determine the running time of an Algorithm

3. Consecutive Statements :

1. **1.** $x = x + 1;$ // **constant c_0 time**
2. **2.** $\text{for}(i=1; i \leq n; i++)$ // **executed n times**
3. **3.** $m = m + 1;$ // **constant c_1 time**
4. **4.** $\text{for}(i=1; i \leq n; i++)\{$ // **executed n times**
5. **5.** $\text{for}(j=1; j \leq n; j++)$ // **executed n times**
6. **6.** $k = k + 1;$ // **constant c_2 time**
7. **7.** $\}$

Determine the running time of an Algorithm

3. Consecutive Statements :

1. **1.** $x = x + 1;$ // **constant c_0 time**
2. **2.** $\text{for}(i = 1; i \leq n; i++)$ // **executed n times**
3. **3.** $m = m + 1;$ // **constant c_1 time**
4. **4.** $\text{for}(i = 1; i \leq n; i++)\{$ // **executed n times**
5. **5.** $\text{for}(j = 1; j \leq n; j++)$ // **executed n times**
6. **6.** $k = k + 1;$ // **constant c_2 time**
7. **7.** $\}$

$$\text{Total time} = c_0 + c_1n + c_2n^2 = O(n^2)$$

Determine the running time of an Algorithm

4. If-then-else Statements :

Worst-case running time:

TEST statement + (***THEN*** part **OR** ***ELSE*** part)

Determine the running time of an Algorithm

4. If-then-else Statements :

1. 1. *if (length() == 0) {*  **TEST** statements
2. 2. *return false;*
3. 3. *} else {*
4. 4. *for(int n=0; n < length(); n++){*
5. 5. *if(!list[n].equals(otherList.list[n]))*
6. 6. *return false;*
7. 7. *}*
8. 8. *}*

Determine the running time of an Algorithm

4. If-then-else Statements :

1. 1. *if (length() == 0) {*  **TEST** statements
2. 2. *return false;*  **THEN** statements
3. 3. *} else {*
4. 4. *for(int n=0; n < length(); n++){*
5. 5. *if(!list[n].equals(otherList.list[n]))*
6. 6. *return false;*
7. 7. *}*
8. 8. *}*

Determine the running time of an Algorithm

4. If-then-else Statements :

```
1. 1. if (length() == 0) {  
2.   return false;  
3. } else {  
4.   for(int n=0; n < length(); n++){  
5.     if(!list[n].equals(otherList.list[n]))  
6.       return false;  
7.   }  
8. }
```

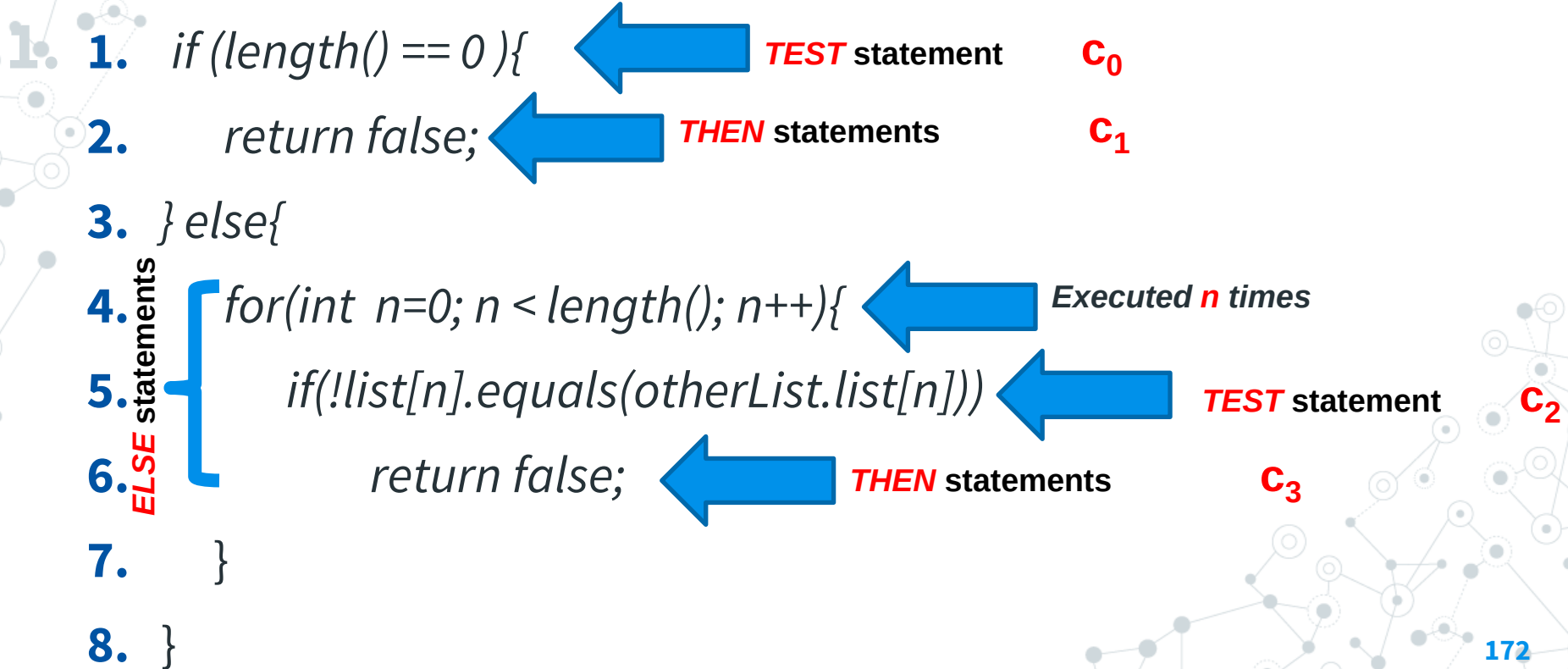
← **TEST** statements

← **THEN** statements

} **ELSE** statements

Determine the running time of an Algorithm

4. If-then-else Statements :



Determine the running time of an Algorithm

4. If-then-else Statements :

```
1. 1. if (length() == 0) {
2.   return false;
3. } else {
4.   for (int n=0; n < length(); n++) {
5.     if (!list[n].equals(otherList.list[n])) {
6.       return false;
7.     }
8.   }

```

Annotations for running time analysis:

- Line 1: **TEST statement** (C_0)
- Line 2: **THEN statements** (C_1)
- Line 4: **Executed n times**
- Line 5: **TEST statement** (C_2)
- Line 6: **THEN statements** (C_3)

ELSE statements (Lines 4, 5, 6)

Total time = $C_0 + (C_2 + C_3) n = O(n)$

Determine the running time of an Algorithm

5. Logarithmic complexity:

if it takes a constant time to cut the problem size by a fraction (usually by $\frac{1}{2}$)

Determine the running time of an Algorithm

5. Logarithmic complexity:

1. *for* ($i=1; i < n;$)

2. $i = i * 2;$

Determine the running time of an Algorithm

5. Logarithmic complexity:

1. *for* ($i=1; i < n;$)

2. $i = i * 2;$

Total time = $O(\log n)$



Thanks!

Any questions?

You can find me at:
aegono@carsu.edu.ph

References :

- ◎ Hanly, J., Koffman, E (2013). Problem Solving and Program Design in C, 7th ed.
- ◎ Karumanchi, N (2017). Data Structures and Algorithms Made Easy. 5th ed.
- ◎ Weiss, M. A (2010) . Data Structures and Algorithm Analysis in C.

Credits

Special thanks to all the people who made and released these awesome resources for free:

- ◎ Presentation template by [SlidesCarnival](#)
- ◎ Photographs by [Unsplash](#)



Thanks!

Any questions?

You can find me at:
aegono@carsu.edu.ph

References :

- ◎ Hanly, J., Koffman, E (2013). Problem Solving and Program Design in C, 7th ed.
- ◎ Karumanchi, N (2017). Data Structures and Algorithms Made Easy. 5th ed.
- ◎ Weiss, M. A (2010) . Data Structures and Algorithm Analysis in C.

Credits

Special thanks to all the people who made and released these awesome resources for free:

- ◎ Presentation template by [SlidesCarnival](#)
- ◎ Photographs by [Unsplash](#)